The Limits of Predictions for Online Bipartite Matching: A Unified Experimental Study and an Impossibility Theorem

[Author Name] — Master's Thesis (draft)

Abstract

Learning-augmented ("with-predictions") algorithms for online bipartite matching have proliferated: MinPredictedDegree consumes a per-node degree predictor, and a family of test-and-fallback schemes tests a type-histogram prediction on a sublinear prefix of the arrivals before committing to follow it or to fall back on an advice-free baseline. These algorithms have been analyzed in isolation. We give the first unified experimental study, placing all of them on a single harness with a common structured prediction-error model, a common optimum, and confidence intervals, across synthetic graphs, six real-world graphs, and real request traces. Our central finding is that on average-case inputs the value of predictions is robustness insurance, not a performance lever: the advice-free baseline is already near-optimal, so the consistency upside of good advice is small, whereas unguarded prediction-following can crash far below the baseline, and the practical worth of the sophisticated algorithms is that they never do. We then prove this wall is necessary: no test-and-fallback algorithm whose test inspects only a sublinear prefix can be both consistent and robust on strong-baseline instances, because the structure that makes the prefix distribution-test feasible is the structure that makes the baseline near-optimal. The proof reduces safe advice-use to tolerant distribution testing and invokes its near-linear sample-complexity barrier. Experiments and theory together deliver one message: where you can test the advice, you do not need it.

Contents

Chapter 1

Introduction

1.1 Background and motivation

Online bipartite matching is the canonical model of irrevocable sequential allocation. Resources are known in advance; requests arrive one at a time; each must be matched to an available compatible resource — or dropped — immediately, before the rest of the input is seen. It is the abstraction behind online advertising, ride-hailing dispatch, organ exchange, and request routing in modern serving systems. Because decisions cannot be undone, the difficulty lies entirely in committing well under uncertainty about the future.

A recent and influential idea is to give such an algorithm a **prediction** about the input — a hint about which resources will be contended, or how many requests of each kind will arrive — and to ask for two guarantees at once: **consistency**, meaning near-optimal performance when the prediction is good, and **robustness**, meaning no worse than a prediction-free algorithm when the prediction is wrong. For online matching specifically, two families of such *learning-augmented* algorithms have appeared: MinPredictedDegree, which consumes a per-resource degree prediction, and a family of *test-and-fallback* schemes, which test a request-type prediction on a short prefix of arrivals before deciding whether to trust it.

These algorithms come with attractive worst-case guarantees, yet — as this thesis demonstrates experimentally — their average-case behavior is strikingly muted: on realistic workloads the sophisticated predictions rarely *help*, while the algorithms' real value is that they do not *hurt*. This gap between the worst-case promise and the average-case reality is the puzzle that motivates this thesis. The algorithms had, moreover, only ever been studied *in isolation* — each on its own input model, against its own notion of prediction error, and largely through theory — so there was no common ground on which to compare them, quantify how much a prediction actually buys, or explain the gap. This thesis provides that common ground and, ultimately, an explanation.

1.2 Research objectives

The thesis is organized around three questions:

1. **How do the learning-augmented online-matching algorithms actually compare**, head to head, under a single prediction-error model and on common inputs — and how much does a prediction help, at what cost, on realistic data?

2. **What are the failure modes of the adaptive test-and-fallback mechanism** — the cost of its test, the calibration of its accept/reject decision, and where it goes wrong?
3. **Why does the average-case experience differ so sharply from the worst-case promise** — is the observed wall an artifact of particular algorithms and generators, or is it *necessary*?

The first two are answered experimentally; the third, theoretically.

1.3 Contributions

- **A unified experimental benchmark** (Chapter 4) that places the advice-free baselines, MinPredictedDegree and its augmentations, and the test-and-fallback algorithms on one harness — common graphs, a common structured prediction-error model, a common optimum, and confidence intervals — the first head-to-head comparison of these families.
- **A characterization of predictions as robustness insurance** (Chapters 4, 7): unguarded prediction-following crashes *below* the advice-free baseline under bad predictions; two distinct mechanisms — structural (the augmentations) and adaptive (test-and-fallback) — restore safety with different consistency/robustness trade-offs; and the consistency upside is small on average-case inputs, so the value is downside protection. The picture is validated on synthetic graphs, six real-world graphs, and real request traces, including with a cheap, non-ML historical predictor.
- **An empirical engagement with the order-error theory** (Chapter 5): MinPredictedDegree's loss is governed by a Kendall-τ order error onto which several error models collapse. We credit Aamand–Chen–Indyk's Appendix D for the order-dependence itself and contribute a characterization of the known bound's looseness and saturation and of the right order measure — not a rediscovery that order matters.
- **The first empirical study of test-and-fallback** (Chapter 6): its testing cost, a threshold-calibration pathology in which a *more accurate* test yields a *worse* decision, its recalibration and the resolution limit that recalibration exposes, and a benchmark of the dynamic combiner revealing an irrevocability penalty that explains why matching requires *test-then-commit*.
- **An impossibility theorem** (Chapter 9): no test-and-fallback algorithm whose test inspects only a sublinear prefix can be both consistent and robust on strong-baseline instances, because the structure that makes the prefix distribution-test feasible is the structure that makes the baseline near-optimal. The proof reduces safe advice-use to *tolerant* distribution testing and invokes its near-linear sample-complexity barrier; it holds for any decision rule, not merely the empirical threshold used in practice. *(The theorem's full proof is complete in architecture — its core lemma rigorous and its construction verified — with one routine step and a final review outstanding; the thesis presents it with its status stated, as is appropriate for in-progress theoretical work.)*

Alongside these, the thesis documents (Chapter 8) the exploratory directions it pursued and the negative results they returned — an attempt to *learn* the predictor for extra performance, and an attempt to find a serving regime where predictions genuinely help — both of which reinforce the central finding and motivated the theorem.

1.4 Thesis organization

Chapter 2 surveys the background: online matching and its input models, the learning-augmented paradigm, the specific prediction-based matching algorithms, and the distribution-testing results the theory relies on. Chapter 3 fixes the model and methodology and validates the experimental harness by reproducing a subset of the Borodin et al. study. Chapter 4 presents the unified benchmark and its four findings. Chapter 5 examines what governs the (small) loss — order error — and its relation to the known bound. Chapter 6 studies the test-and-fallback mechanism in depth. Chapter 7 tests external validity on real predictors and real graphs. Chapter 8 reports the exploratory directions and negative results. Chapter 9 proves the impossibility theorem. Chapter 10 presents the AI-inference serving case study. Chapter 11 concludes and discusses future work. The recurring thesis, established experimentally and then proven necessary, is a single sentence: **on average-case online matching, predictions are robustness insurance rather than a performance lever — and where you can test the advice, you do not need it.**

Chapter 2

Background and Related Work

This chapter sets up the technical context the thesis builds on: the online bipartite-matching problem and its input models (§2.1), the known-i.i.d. model we work in (§2.2), the learning-augmented ("with-predictions") paradigm and its consistency/robustness language (§2.3), the specific prediction-based matching algorithms we benchmark and bound (§2.4), the distribution-testing results our impossibility theorem relies on (§2.5), and the gaps in the literature this thesis addresses (§2.6).

2.1 Online bipartite matching and competitive analysis

In online bipartite matching, one side of a bipartite graph — the *offline* resources — is known in advance, while the vertices of the other side — the *online* requests — arrive one at a time. On each arrival the algorithm sees the request's incident edges and must either match it to a currently unmatched neighbor or leave it unmatched, **immediately and irrevocably**. The goal is to maximize the size (or, in weighted variants, the value) of the matching. Performance is measured by the **competitive ratio**, the (expected) ratio between the algorithm's matching and an offline optimum on the same input.

The difficulty of the problem depends entirely on the assumed *input model*:

- **Adversarial arrival order.** The requests and their order are chosen by an adversary. The seminal result of Karp, Vazirani and Vazirani [1] is that the randomized **Ranking** algorithm — fix a uniformly random priority order on the resources and match each request to its highest-priority available neighbor — achieves competitive ratio $1 - 1/e \approx 0.632$, and that this is optimal for the adversarial model. Deterministic algorithms cannot beat $1/2$.
- **Random arrival order.** The graph is adversarial but the requests arrive in a uniformly random order; this is strictly easier than adversarial order and admits ratios above $1 - 1/e$.
- **Known-i.i.d.** Requests are drawn independently from a *known* distribution over request types (§2.2); this is the average-case model and the one this thesis studies.

Because the known-i.i.d. and random-order models are more benign than adversarial order — formally, Known-I.I.D. $\leq$ Random-Order in difficulty — algorithms and guarantees designed for the harder models carry over, a fact we use throughout.

2.2 The known-i.i.d. model

In the known-i.i.d. model there is a bipartite *type graph*: each online **type** ℓ has a fixed neighborhood $N(\ell)$ among the offline resources, and an instance is generated by drawing m arrivals independently from a known distribution p over types. The type graph and p are known; the realized sequence is not. This models settings — ad serving, recurring request streams — where the population of requests is statistically stable even though individual arrivals are not.

A line of work has pushed the worst-case (over type graphs) competitive ratio above the $1 - 1/e$ adversarial barrier: Feldman et al. [2] first beat it (0.67) via a flow-based *blue/red* decomposition of a suggested matching; Manshadi, Oveis Gharan and Saberi [3] and Jaillet–Lu [4] improved the ratio (to ≈ 0.702 and ≈ 0.729 respectively) using LP-based and list-based online policies; subsequent work refined it further. These are the "sophisticated" algorithms whose worst-case optimality motivates their use.

Crucially for this thesis, Borodin, Karavasilis and Pankratov [5] conducted an experimental study of these algorithms and found that on *average-case* i.i.d. instances the picture is very different from the worst case: the simple algorithms (Greedy, Ranking) perform almost as well as the sophisticated ones, whose advantage is a worst-case, not a typical-case, phenomenon. This empirical observation — that on typical inputs the simple baseline is already near-optimal — is the seed of the thesis's central finding, and we reproduce a subset of it as validation (Chapter 3).

2.3 Learning-augmented algorithms

The **algorithms-with-predictions** (or *learning-augmented*) paradigm, surveyed in [6], equips an online algorithm with a prediction about the unknown input and asks for two guarantees simultaneously:

- **Consistency:** near-optimal performance when the prediction is accurate;
- **Robustness:** performance no worse than a prediction-free guarantee when the prediction is arbitrarily wrong.

The paradigm was crystallized by Lykouris and Vassilvitskii [7] for competitive caching, who showed how to interpolate between trusting and ignoring a predictor while bounding both consistency and robustness. A large literature followed across ski rental, scheduling, and other online problems, including the *optimal* consistency/robustness trade-off analyses of Wei and Zhang [8], which establish problem-intrinsic Pareto frontiers between the two objectives. A recurring theme is that robustness is obtained by *hedging* — combining the predictor's advice with a safe default so that a bad prediction cannot cause catastrophe. The blind-follow-with-switching **combiner** of Chłędowski, Polak, Szabucki and Żołna [9], introduced in an experimental study of robust learning-augmented caching, is one such hedging mechanism; we port and benchmark it (Chapter 6). That paper is also the closest methodological template for this thesis's experimental half. Recent theoretical work further analyzes how the achievable ratio degrades *continuously* with prediction accuracy in a distributionally-robust formulation [10], complementing the discrete consistency/robustness endpoints used here.

2.4 Predictions for online bipartite matching

Two strands apply the with-predictions paradigm to online matching, differing in the *prediction object* they consume.

Degree predictions: MinPredictedDegree. Aamand, Chen and Indyk [11] propose **MinPredictedDegree (MPD)**: given a prediction μ of each offline resource's degree (how contended it will be), match each arrival to its available neighbor of *minimum predicted degree* — i.e. protect the resources predicted to be rarest. MPD is robust by construction: a constant (useless) predictor reduces it to Ranking. A key structural fact, which the thesis engages in Chapter 5, is that MPD depends on μ *only through the order it induces*; the authors' Appendix D bounds the matching loss by an order quantity ($n - \mathrm{LIS}$, the number of resources not in the longest non-decreasing subsequence of the true degrees ordered by μ), and shows in particular that a monotone (order-preserving) prediction incurs zero loss.

Type-histogram advice and test-and-fallback. A second strand takes the prediction to be a *histogram* $\hat{c}$ over request types (how many of each type will arrive). Choo et al. [12] introduce **TestAndMatch**: build a matching from $\hat{c}$ and Mimic it, but first spend a sublinear prefix of arrivals *testing* whether the observed type frequencies match $\hat{c}$ — using an ℓ_1-distance tester adapted from Jiao, Han and Weissman [13] — and fall back to Ranking if they do not. Burathep, Erlebach and Moses [14] generalize this ("Test-and-Match+") to the random-arrival model and to imperfect knowledge of the matching size. Both papers give *upper* bounds (algorithms); their only lower bound (Choo et al.'s Theorem 3.1) is a generic *adversarial* indistinguishability result (no algorithm is 1-consistent and $> \frac{1}{2}$-robust), and neither proves a lower bound in the stochastic model. Notably, Choo et al.'s acceptance threshold already *couples* to the baseline competitive ratio β — but constructively, inside the algorithm design, not as an impossibility. Turning that coupling into a two-sided, algorithm-independent impossibility is the theoretical contribution of this thesis (Chapter 9).

2.5 Distribution testing

The thesis's impossibility theorem reduces the problem of *safely using* histogram advice to a question in **distribution testing**: given samples from an unknown distribution p over a support of size r and a known reference q, decide how far p is from q in ℓ_1 (total-variation) distance. Two regimes must be distinguished:

- **Identity (non-tolerant) testing** — distinguish $p = q$ from $\|p - q\|_1 \geq \varepsilon$ — has sample complexity $\Theta(\sqrt{r}/\varepsilon^2)$ [15, 16], *sublinear* in the support size.
- **Tolerant testing / distance estimation** — distinguish $\|p - q\|_1 \leq \varepsilon_1$ from $\geq \varepsilon_2$ for $0 < \varepsilon_1 < \varepsilon_2$, i.e. estimate the distance rather than test equality — is provably *much harder*: Valiant and Valiant [17] showed it requires $\Theta(r/\log r)$ samples, *near-linear* in the support. Jiao, Han and Weissman [13] gave matching bounds for ℓ_1-distance estimation, and Canonne, Jain, Kamath and Li [18] precisely characterized the whole $(\varepsilon_1, \varepsilon_2)$ landscape, showing that for constant tolerances the cost jumps to the "barely sublinear" $\tilde{\Theta}(r/\log r)$.

The near-quadratic gap between $\sqrt{r}$ (testing) and $r/\log r$ (tolerant testing) is the technical engine of this thesis's theorem: safely following advice requires distinguishing "close enough to help" from "far enough to hurt" — a *tolerant* test — and it is exactly this near-linear sample cost that a sublinear prefix cannot pay. To our knowledge, the connection between distribution-testing sample

complexity and the *value of advice in an online algorithm* has not been made before.

2.6 Positioning of this thesis

Against this background, three gaps stand out, which the thesis addresses in turn:

1. **No unified empirical comparison.** The matching algorithms above were each studied in isolation, on their own input families and error models, and largely in theory; there is no head-to-head experimental benchmark under a common harness. (Chapters 4–7.)
2. **No empirical study of test-and-fallback.** The distribution test at the heart of [12, 14] has no deployable implementation (the authors themselves fall back to an empirical surrogate), and its testing cost, threshold calibration, and failure modes have not been measured. (Chapter 6.)
3. **No lower bound coupling testability to baseline strength.** No prior work proves that a sublinear-test algorithm cannot be both consistent and robust on strong-baseline instances, nor identifies the few-types structure that makes the test feasible with the structure that makes the baseline near-optimal. (Chapter 9.)

The thesis closes the first two gaps experimentally and the third theoretically, arriving at a single unifying statement — *on average-case online matching, predictions are robustness insurance rather than a performance lever, and this is necessary* — supported by experiments across synthetic graphs, real graphs, and real traces, and by an impossibility theorem.

Chapter 3

Model, Algorithms, and Methodology

Chapter 2 placed the problem in its field. This chapter fixes the precise model and notation used throughout (§3.1), details the algorithms as we implement them (§3.2), the prediction objects and structured error models (§3.3), the consistency/robustness measures (§3.4), and the experimental harness (§3.5). It closes by validating the harness against the published Borodin et al. figures before any contribution is built on it (§3.6).

3.1 The known-i.i.d. model, precisely

There is a set R of n offline **resources** and a **type graph** on r online **types**; type ℓ has neighborhood $N(\ell) \subseteq R$. An instance is generated by drawing m arrivals i.i.d. from a distribution p over the r types; the i-th arrival reveals its type (hence $N(\cdot)$) and must be matched to a currently unmatched resource in its neighborhood, immediately and irrevocably, or left unmatched. The type graph and p are known to the algorithm; the realized arrival sequence is not. Following the standard *integral-types* convention, the default is $m = n$ with uniform p, so each type's expected count is an integer; the classical setting has $r = n$ (each offline vertex a distinct type), and the *few-types* setting used for the histogram-advice experiments has $r \ll n$.

The performance measure is the **competitive ratio** $\rho(\mathrm{ALG}) = \mathbb{E}[|\mathrm{ALG}|]/\mathbb{E}[|\mathrm{OPT}|]$, where OPT is a maximum matching of the *realized* instance, computed exactly by Hopcroft–Karp [19]. The advice-free baseline throughout is KVV Ranking, whose ratio we write ρ_{base} (the **baseline strength**).

3.2 Algorithms

All greedy-type algorithms are instances of one primitive: given a total order (**rank**) on the resources, **GreedyWithPermutation** matches each arrival to its available neighbor of minimum rank. This makes the algorithm family a single code path parameterized by the rank, which is important both for a fair comparison and for the theory.

- **Advice-free baselines.** SimpleGreedy (identity rank); **Ranking** (uniformly random rank; the baseline); and the stochastic-matching algorithms Feldman and Jaillet–Lu, which precompute a suggested matching by max-flow — Feldman via a cap-$\{2, 1, 2\}$ flow network and a blue/red decomposition of the unit-flow subgraph, Jaillet–Lu via a cap-$\{3, 2, 3\}$ network yielding a fractional solution $f^\star \in \{0, \frac{1}{3}, \frac{2}{3}\}$ and per-type list distributions. Each has a *non-greedy*

variant (follow the suggestion only) and a *greedy* variant (fall back to any available neighbor).

- **Degree predictions.** MinPredictedDegree (MPD) is GreedyWithPermutation ranked by ascending predicted degree $\mu \in \mathbb{R}^R$. Constant μ gives Ranking; the true realized degrees give the consistency ceiling (MinDegree). The **augmentations** Feldman(MPD) and JailletLu(MPD) use the MPD rank only as the tie-break of the greedy fallback, so the worst-case-optimal base matching carries the load.
- **Type-histogram advice and test-and-fallback.** From a count vector $\hat{c}$ over types we build an advice matching $\hat{M}$ (a maximum matching of $\hat{c}$ copies) and record its per-type partners. **FollowPrediction** *mimics* $\hat{M}$ — routes every arrival to its type's advice partner; **TestAndMatch** (Choo and BEM variants) *mimics* over a length-k prefix, tests the empirical ℓ_1 distance between the prefix type frequencies and $q = \hat{c}/\|\hat{c}\|_1$ against a threshold τ, then continues or falls back to Ranking. We also port the Chłędowski-style dynamic **combiner** as a benchmarked baseline (not a contribution).

3.3 Prediction objects and structured error models

The families consume different prediction objects — degree vectors μ and type histograms $\hat{c}$ — so each is corrupted by its own knob and reported in a parallel panel. For μ we use four structured error models, each returning both an ℓ_1 *magnitude* and a normalized **Kendall-τ order error**: random-flip, systematic-bias (a monotone rescale — order-preserving, hence Kendall-$\tau \equiv 0$ by construction), adversarial (reflection of a fraction of entries), and distribution-drift (blend toward an independently drawn graph's degrees). For $\hat{c}$ we blend the realized counts toward a concentrated random target by a fraction $\eta \in [0, 1]$, reporting the induced $\ell_1(p, q)$. Because MPD depends on μ only through the induced order, the order/magnitude distinction is the axis of Chapter 5.

3.4 Consistency and robustness

For a prediction-consuming algorithm, **consistency** is its ratio under perfect advice and **robustness** is its worst-case ratio under adversarial advice. A useful algorithm is consistent *and* never (much) below ρ_{base}; the tension between the two is the subject of Chapters 6 and 9.

3.5 The experimental harness

The harness is a small, dependency-light Python codebase (NumPy, SciPy, NetworkX). All randomness comes from independent, reproducible streams obtained by NumPy's spawn mechanism — by default four streams (graph, instance, Ranking, Jaillet–Lu), with additional streams spawned for prediction generation and perturbation, so adding an algorithm never disturbs existing results. We use **paired trials**: within a comparison, every algorithm and error level reuses the same graph, arrival sequence, OPT, and tie-break seed, so differences are attributable to the prediction alone. Reported ratios are means over trials with 95% normal-approximation confidence intervals. Every figure and table in the thesis is regenerated from a fixed seed by a single script (Appendix A).

3.6 Fidelity check: reproducing Borodin et al.

Before building prediction-based algorithms on the harness, we validated it against the published experimental study of Borodin, Karavasilis and Pankratov [5]. This is a **fidelity check, not a contribution**: its purpose is to confirm that the generators, the i.i.d. sampler, the Hopcroft–Karp optimum, and the algorithm implementations reproduce the paper's qualitative findings, so that later differences can be attributed to the algorithms rather than to infrastructure. Our stack (Python/NetworkX) differs from the paper's (C++/Edmonds–Karp), so we target *qualitative* agreement, accepting small absolute differences (≤ 0.02).

We reproduced the core four algorithms (six greedy/non-greedy variants) on two random graph families at $n = 1000$, $m = n$, 100 trials per parameter value (matching the paper's setup), under a single master seed.

Erdős–Rényi (edge probability c/n; the paper's Fig. 9, partial). Sweeping c reproduces the characteristic U-shape and its hard case: SimpleGreedy attains its minimum 0.864 at $c \approx 4.9$, and the greedy variants of the complex algorithms their minima ≈ 0.884 at $c \approx 5.3$. Ranking is indistinguishable from SimpleGreedy (maximum difference 0.0017 across all 75 values — the paper omits Ranking's curve for exactly this reason), and the non-greedy variants degrade monotonically as c grows ($0.987 \to 0.729$ for Feldman, $0.985 \to 0.764$ for Jaillet–Lu). Every one of the paper's five prose claims we checked holds.

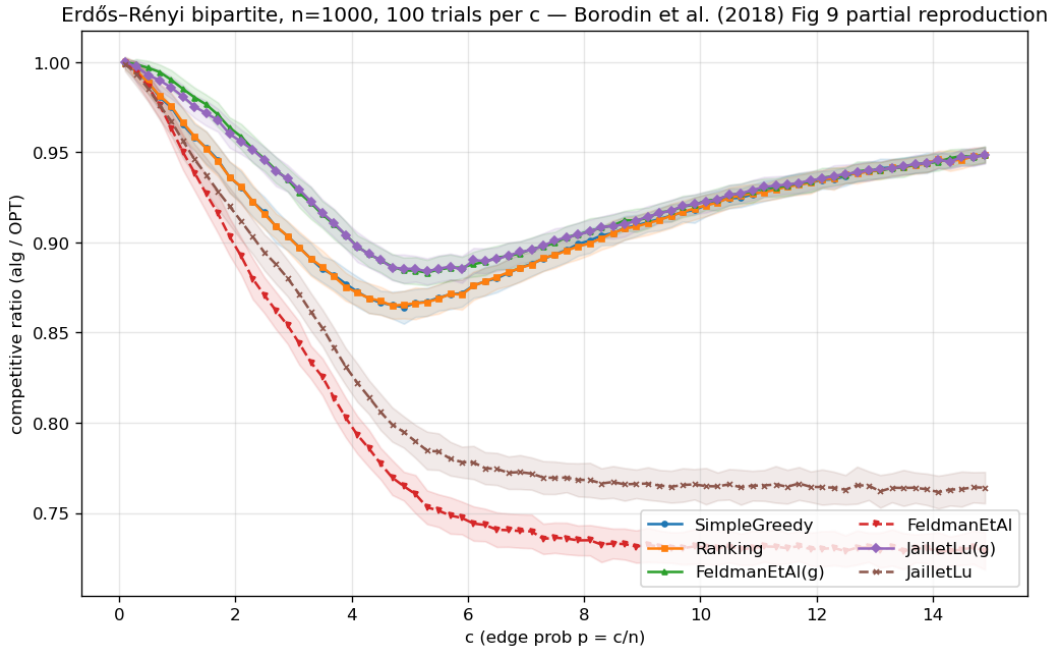

Figure 3.1: Erdős–Rényi reproduction (competitive ratio vs edge density c): the characteristic U-shape, greedy minima near $c \approx 4.9$, and non-greedy degradation as c grows.

Random left-regular (left-degree d; the paper's Fig. 18, partial). The pattern repeats with the hard case at $d = 5$ (SimpleGreedy minimum 0.890), Ranking $\approx$ SimpleGreedy (max difference 0.005), non-greedy degradation as d grows, and greedy complex variants $\approx$ SimpleGreedy asymptotically.

A cross-family observation. Combining the two sweeps surfaces a non-obvious fact: the non-

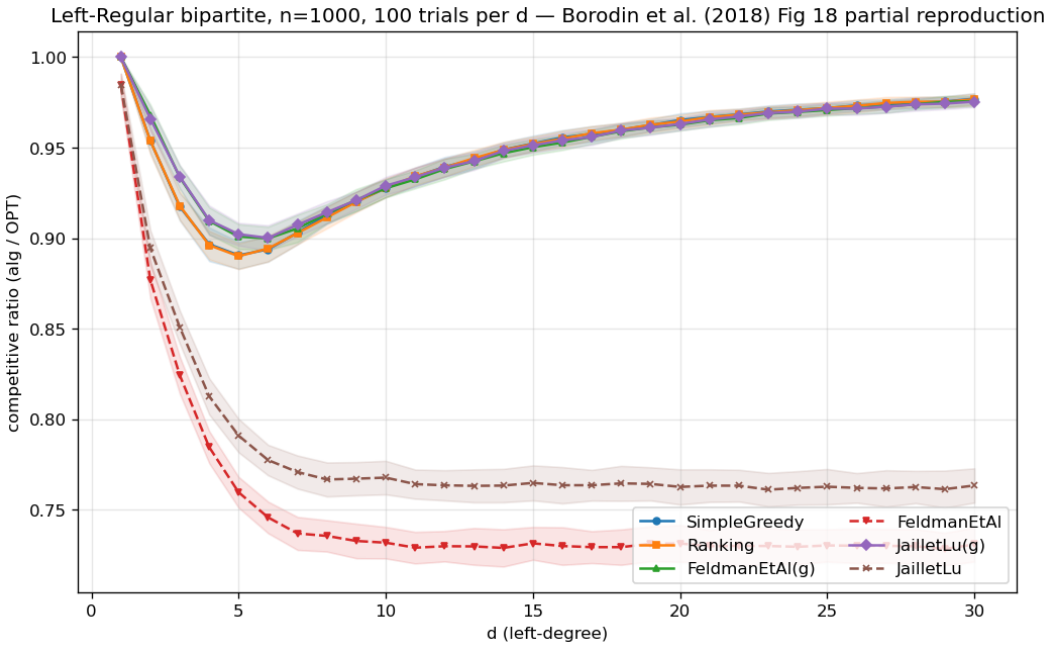

Figure 3.2: Random left-regular reproduction (competitive ratio vs left-degree d): the hard case at $d = 5$, Greedy $\approx$ Ranking, non-greedy degradation.

greedy Feldman and Jaillet–Lu algorithms converge to the *same* asymptotic ratio in *both* families — 0.729 and 0.764 respectively, within 0.002 across families — and both sit *above* their worst-case theoretical bounds (0.670 and 0.729) by +0.06 and +0.03. This hints at a universal "average-case asymptotic constant" distinct from the worst-case guarantee — an early, concrete instance of the thesis's recurring theme that the average case is far more benign than the worst case, which the prediction-error sweeps of later chapters probe in earnest.

Verdict. The harness reproduces the published qualitative findings on both families, including the locations of the hard cases and the near-identity of Greedy and Ranking. The foundation is trusted; the contributions of Chapters 4–9 are built on it. (Full reproduction tables and the paper-claim checklist are in Appendix A.)

Chapter 4

The Unified Benchmark

Having validated the harness (Chapter 3), we now place all three algorithm families on it and establish the thesis's organizing finding. This chapter reports competitive ratios (mean ± 95% CI) as a function of prediction quality, in three panels chosen to span the regimes each family was designed for; the four findings it draws out (§4.2) recur through the rest of the thesis.

4.1 Design

The families consume different prediction objects, so each is driven by its own corruption knob and reported in a parallel panel (Chapter 3); the shared harness — graphs, OPT, CI methodology, and the advice-free floor — is what makes the panels comparable.

- **Panel A — clvb_zipf** (n = 1000, 60 trials): heavy-tailed offline degrees, where degree predictions carry signal.
- **Panel B — left-regular** d=5 (n = 1000, 60 trials): near-homogeneous degrees, where predictions add little.
- **Panel C — few-types** r=8 (n = 2000, 50 trials): near-perfect-matchable, the calibrated regime for the type-histogram test-and-fallback algorithms.

For the degree-prediction panels the quality columns are *perfect* (true degrees), *noisy* (random-flip at strength $\frac{1}{2}$), *adversarial* (order-reversing), and *garbage* (fully random ≡ Ranking). For the advice panel they are $\eta \in \{0, 0.3, 0.6, 1.0\}$ (*perfect / mild / bad / garbage*). Confidence intervals are tight throughout (±0.001–0.003). **Table 4.1** and **Figure 4.1** are the chapter's data; the salient rows:

	perfect	noisy	adversarial	garbage
Panel A — clvb_zipf				
Ranking (floor)	0.948	—	—	—
MinDegree (oracle)	0.996	—	—	—
MPD	0.989	0.956	**0.908**	0.946
Feldman(MPD)	0.981	0.979	0.976	0.978
JailletLu(MPD)	0.977	0.976	0.974	0.975

	perfect	noisy	adversarial	garbage
Feldman / JailletLu (base)	0.887 / 0.901	—	—	—
Panel B — left-regular d=5				
Greedy = Ranking (floor)	0.890	—	—	—
MinDegree (oracle)	0.966	—	—	—
MPD	0.932	0.906	**0.854**	0.888
Feldman(MPD) / JailletLu(MPD)	0.906 / 0.903	0.903 / 0.902	0.896 / 0.898	0.900 / 0.901
Feldman / JailletLu (base)	0.758 / 0.789	—	—	—

	perfect	mild	bad	garbage
Panel C — few-types r=8				
Ranking (floor)	0.990	—	—	—
MinDegree (oracle)	0.999	—	—	—
FollowPrediction	1.000	0.832	0.679	**0.472**
TestAndMatch (Choo)	1.000	0.984	0.989	0.990
TestAndMatch (BEM)	0.998	0.988	0.988	0.968
Combiner *(benchmark)*	0.990	0.990	0.990	0.990

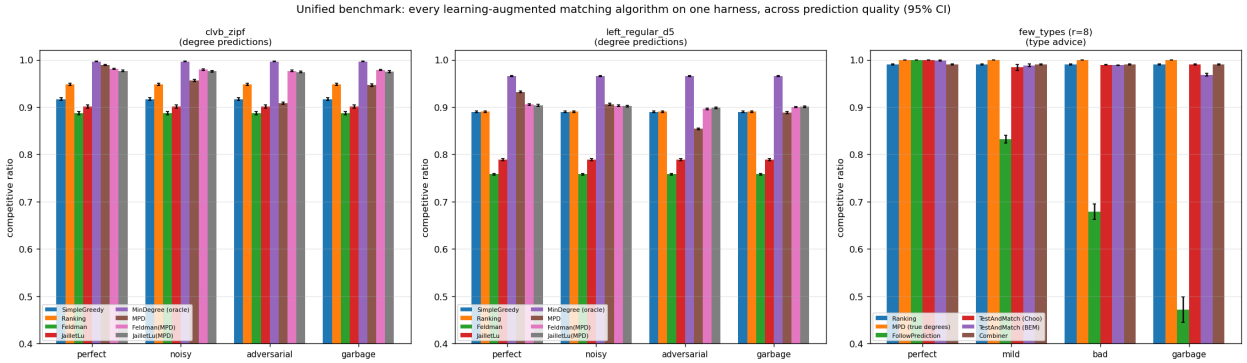

Figure 4.1: The unified benchmark: competitive ratio (mean, 95% CI) versus prediction quality, for the three panels. Naive followers (MPD under adversarial, FollowPrediction) dip below the advice-free floor; the robust algorithms do not.

4.2 Four findings

(F1) Robustness is engineered, not free: naive followers crash below the floor. Both unguarded prediction-followers dive under the advice-free Ranking floor once the prediction is adversarial or garbage: MPD falls to $0.908 < 0.948$ (Panel A) and $0.854 < 0.890$ (Panel B), and FollowPrediction collapses to $0.472 \ll 0.990$ (Panel C). Using either *unguarded* is strictly worse than using no prediction at all. Every robust algorithm — the augmentations, TestAndMatch, the combiner — avoids this by construction.

(F2) Two distinct robustness mechanisms, with different shapes. *Structural* robustness (Feldman(MPD), JailletLu(MPD)): the worst-case-optimal base matching carries the load and the prediction only breaks ties, so performance is nearly *flat* — Feldman(MPD) moves only $0.981 \to 0.976$ from perfect to adversarial (Panel A). It cannot crash but caps the upside (never reaching the 0.996 oracle). *Adaptive* robustness (TestAndMatch): test a sublinear prefix, then commit — capturing the upside when advice is good (Choo 1.000) and holding the floor when it is bad (0.990). On Panel C it is the only algorithm on the upper envelope at both ends. The two mechanisms trade consistency for robustness in opposite ways; **Figure 4.2** plots every algorithm on the consistency–robustness plane, where the opposite trades — and the empty region beyond TestAndMatch toward the ideal top-right corner — are visible at a glance.

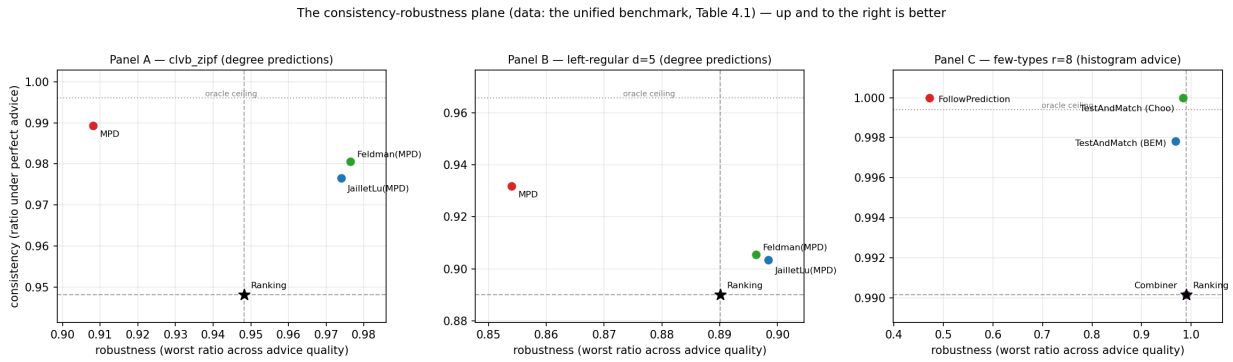

Figure 4.2: The consistency–robustness plane, one point per algorithm (consistency = ratio under perfect advice; robustness = worst ratio across advice quality; the data of Table 4.1). Dashed lines mark the advice-free floor, dotted the oracle ceiling. Naive followers (MPD, FollowPrediction) buy consistency with robustness; the structural augmentations give up little of either; TestAndMatch sits nearest the ideal top-right corner; the combiner stays pinned at the floor point.

(F3) The consistency upside is small on average-case inputs; the spread lives on the bad-advice side. On few-types the advice-free Ranking is already 0.990 and MPD-with-true-degrees is 0.999 — under 0.01 for any advice to add on the good side. Every wide gap in Panel C is a *downside* gap. This is the thesis in one panel; Chapter 9 shows it is *necessary*.

(F4) The augmentation rescues structurally weak base algorithms. Feldman and Jaillet–Lu are tuned for the worst-case ratio and are the *weakest* advice-free entries on these average-case inputs (Panel B: 0.758 / 0.789, below Greedy's 0.890); the MPD augmentation lifts them to ≈ 0.90. The prediction does *more* for the worst-case-designed algorithms than for greedy — a pairing visible only under a unified table.

4.3 Chapter summary

On average-case matching the advice-free baseline is already near-optimal, unguarded prediction-following is unsafe, and the value of the sophisticated algorithms is downside protection delivered by one of two mechanisms. Chapters 5–7 sharpen each part — what governs the (small) loss, what the adaptive test costs, and whether the picture survives on real data — and Chapter 9 proves the wall is a theorem, not an artifact.

Chapter 5

What Governs the Loss: Order Error and ACI's Bound

Chapter 4 showed that on average-case inputs the loss of a degree-prediction algorithm is small. This chapter asks what *governs* that loss. MinPredictedDegree matches by ascending predicted degree, so it depends on the predictor μ *only through the order it induces* on the resources: two predictors inducing the same order produce identical matchings, and a monotone rescaling of μ changes nothing. The right question is therefore not "how large is the error" but "which *order* error governs the loss, and how tightly." Answering it requires care, because the qualitative fact that order — not magnitude — matters is already a theorem, and we are explicit about what is prior and what is ours.

5.1 What is already known (ACI)

Aamand, Chen and Indyk [11, Appendix D] prove that on the CLV-B model, MinPredictedDegree's matching loss relative to the true expected degrees is at most $n - \mathrm{LIS}(p[\mu])$, where $p[\mu]$ is the true weights ordered by μ and LIS is the longest non-decreasing subsequence — a pure *order* quantity. In particular a monotone (order-preserving) predictor has $p[\mu]$ already sorted, so $n - \mathrm{LIS} = 0$ and the loss is zero. Thus *order-dependence* and *the zero-effect of a monotone bias* are ACI's results, not ours; our `systematic_bias` error model (a monotone rescale) has Kendall-$\tau \equiv 0$ by construction and, consistently, leaves MPD's ratio exactly unchanged across the benchmark (Chapter 4) — an empirical confirmation of ACI's statement, not a new finding.

5.2 Our characterization

We sweep the four structured error models across strength and record, per model and level on the same instances, three quantities: the actual MPD matching loss, ACI's $n - \mathrm{LIS}$, and the normalized Kendall-τ order error (**Figure 5.1**; $n = 1000$, Zipf exponent 1.0, 40 trials).

(i) ACI's $n - \mathrm{LIS}$ bound is correct but very loose. The realized loss lies far below the bound at every point — by roughly 16× (adversarial) to 75× (distribution-drift); all points hug the axis in Figure 5.1(a).

(ii) $n - \mathrm{LIS}$ saturates and cannot distinguish error structures. For every non-trivial error

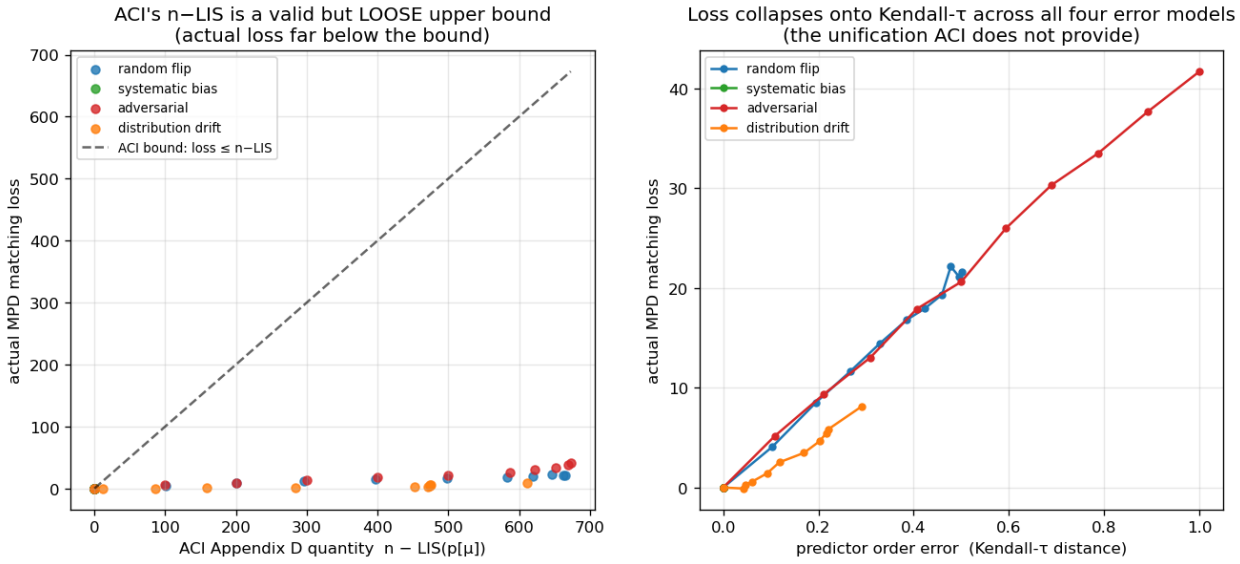

Figure 5.1: Order error governs the loss. (a) The realized MPD loss lies far below ACI's $n - \text{LIS}$ upper bound and $n - \text{LIS}$ saturates. (b) The loss collapses onto the Kendall-τ order error across all four error models.

it is pinned near its maximum ($\approx$ 610–673 for $n = 1000$), even though the true losses differ by a factor of five across models (8.1 drift, 21.6 random-flip, 41.7 adversarial). As a bound that is almost always $\approx n$, it carries little information about which prediction is more harmful.

(iii) Kendall-τ is the governing order measure, and the models collapse onto it. Plotted against Kendall-τ (Figure 5.1(b)), the four models fall on one increasing curve — loss rises with τ (0.29 $\rightarrow$ 0.50 $\rightarrow$ 1.0 for drift, random-flip, adversarial, tracking 8.1 $\rightarrow$ 21.6 $\rightarrow$ 41.7), with `systematic_bias` pinned at $\tau = 0$, zero loss. The quantity that *predicts* the loss and unifies the error structures is the Kendall-τ order distance, which the saturated $n - \text{LIS}$ cannot resolve.

5.3 Chapter summary

We do not claim that order rather than magnitude governs MPD — ACI's Appendix D establishes that, along with the zero-effect of a monotone bias. What we add is a precise empirical characterization: ACI's $n - \text{LIS}$ bound is loose by one to nearly two orders of magnitude and saturates, whereas Kendall-τ predicts the realized loss and unifies the four error structures. This both sharpens the theory's practical content and justifies the single order-error axis used when the predictor is stressed on real data (Chapter 7).

Chapter 6

Test-and-Fallback in Depth

The test-and-fallback algorithms are the adaptive robustness mechanism of Chapter 4 and the object of the theory in Chapter 9. This chapter gives their first empirical study: the robustness envelope they achieve (§6.1), a counter-intuitive failure of the acceptance threshold (§6.2), its recalibration and the resolution limit that recalibration exposes (§6.3) — the empirical face of the impossibility we prove in Chapter 9 — and a benchmark of the dynamic combiner that explains the *commit-once* structure (§6.4). Throughout, the ℓ_1 test is the empirical surrogate the original authors also fall back to (Chapter 3).

6.1 The robustness envelope

Sweeping advice error $\ell_1(p, q)$ on few-types instances ($n = 2000$, $r = 8$, prefix $k = 200$, 40 trials; **Figure 6.1**), FollowPrediction degrades *linearly* from 1.000 at perfect advice to 0.453 at $\ell_1 \approx 1.1$ — well *below* the advice-free floor (≈ 0.99). TestAndMatch instead stays on the **upper envelope**: it captures the benefit when advice is good and, when advice is bad, its prefix test rejects it and it falls back to Ranking, never crashing (BEM $0.998 \to 0.969$; Choo $1.000 \to 0.991$). This is the adaptive counterpart of the structural robustness of Chapter 4.

6.2 A threshold that is too lenient on average-case inputs

Sweeping the prefix (testing) size k at *borderline* advice ($\eta = 0.15$, true $\ell_1 \approx 0.16$) — where the test works hardest (**Figure 6.2**) — a larger, more accurate test makes the *worse* decision: as k grows $25 \to 800$, the ratio *falls* $0.992 \to 0.956$ and the misjudgement rate *rises* $0.00 \to 0.60$. The mechanism: the Choo/BEM threshold τ is calibrated to the worst-case baseline $\beta \approx 0.696$, but on these instances the baseline is ≈ 0.99 (F3), so the empirical break-even sits at $\ell_1 \approx 0$, far below τ. A small noisy prefix over-estimates ℓ_1 and *accidentally rejects* the borderline advice (landing safely on the floor); a large accurate prefix correctly measures $\ell_1 \approx 0.16 < \tau$ and *accepts* the mildly-bad advice the worst-case threshold deems acceptable — underperforming the baseline. On strong-baseline inputs the worst-case threshold is too lenient, and a more accurate test only follows it more faithfully.

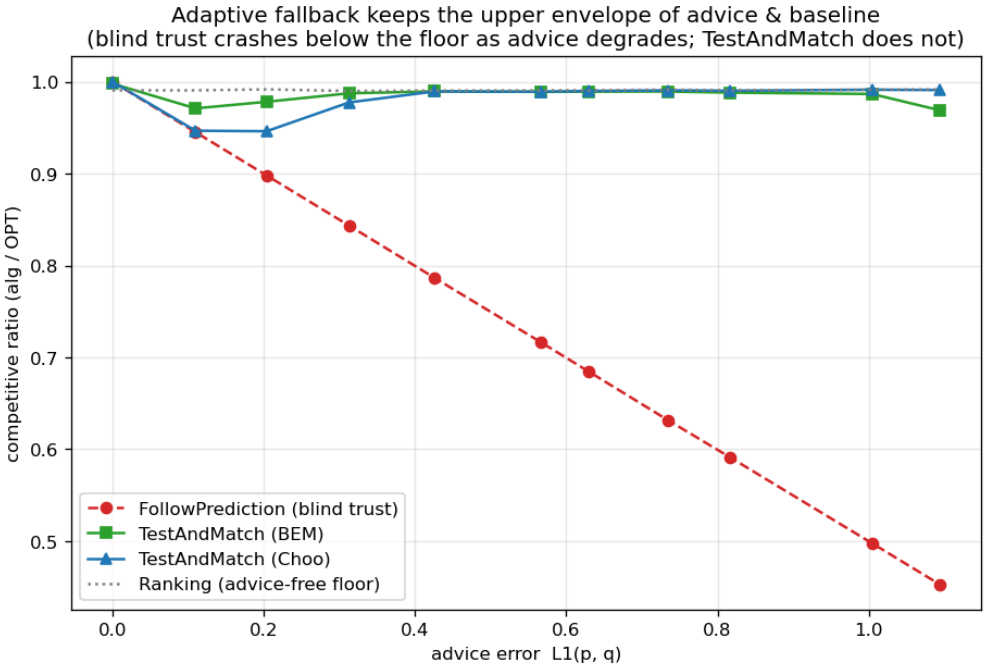

Figure 6.1: The robustness envelope. As advice error $\ell_1(p, q)$ grows, blind FollowPrediction crashes below the advice-free floor, while TestAndMatch (Choo/BEM) stays on the upper envelope.

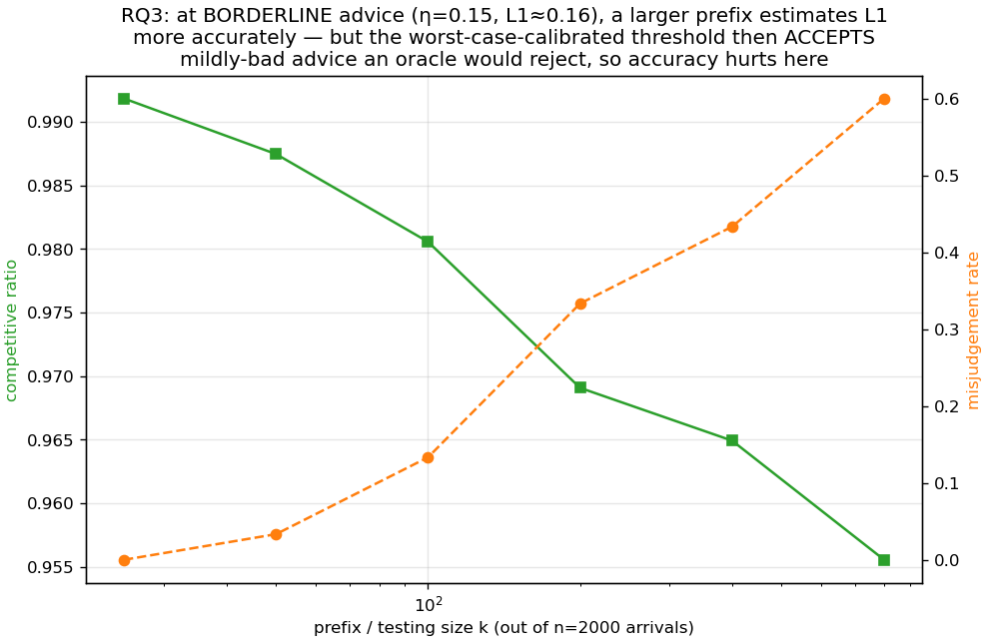

Figure 6.2: Testing cost at borderline advice: a larger, more accurate prefix test makes the *worse* decision, because the worst-case-calibrated threshold accepts mildly-bad advice on strong-baseline inputs.

6.3 Recalibration, and the resolution limit it exposes

Recalibrating τ to the *measured* baseline $\hat{\beta}$ eliminates the pathology (**Figure 6.3**): at borderline advice the worst-case threshold's misjudgement climbs $0.03 \to 1.00$ as the prefix grows and its ratio drops to 0.920, while the recalibrated threshold holds misjudgement 0.00 at every prefix and ratio ≈ 0.986. But recalibration exposes a deeper limit. At perfect advice the worst-case threshold scores 1.000 while the recalibrated one scores only 0.987: the recalibrated $\tau \approx 2(1-\hat{\beta}) \approx 0.028$ is *smaller than the empirical-ℓ_1 estimator's noise floor* (≈ 0.05–0.13), so it can never confidently *accept* — it rejects everything, including perfect advice, and always plays the baseline. In short:

> On strong-baseline instances, no practical empirical-ℓ_1 threshold can both capture the consistency upside and stay safe: the upside is tiny and sits *below the estimator's resolution*. The worst-case threshold over-accepts; the recalibrated one over-rejects; a better tester only follows whichever threshold more faithfully.

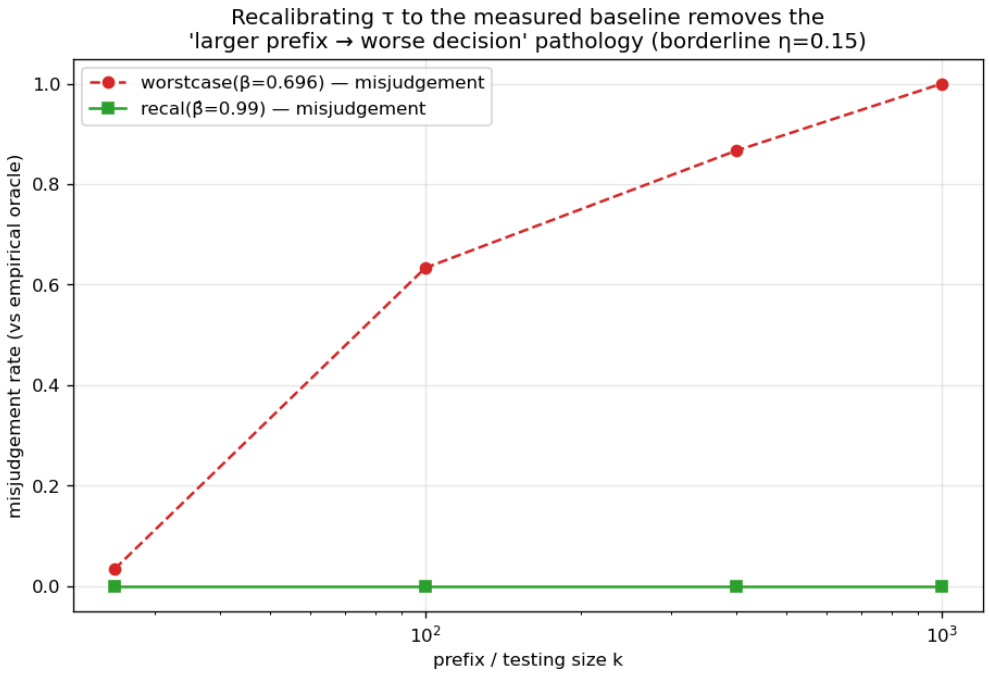

Figure 6.3: Recalibration removes the threshold pathology: at borderline advice the worst-case-calibrated threshold's misjudgement rate climbs toward 1.0 as the prefix grows, while the threshold recalibrated to the measured baseline holds it at zero — but its acceptance margin then falls below the estimator's noise floor, so it can never confidently accept.

This is exactly the phenomenon Chapter 9 proves unavoidable — there, for *any* test on a sublinear prefix, not only the empirical-ℓ_1 threshold. This chapter is its empirical shadow; Chapter 9 is the theorem.

6.4 The dynamic combiner is dominated, and shows why matching needs test-then-commit

We benchmark the Chłędowski-style dynamic combiner to contextualize the commit-once structure of test-and-fallback. In its robust tuning the combiner sits exactly on the floor (0.990 across all advice quality): it never crashes but captures none of the consistency upside, strictly dominated by TestAndMatch. More instructively, an *eager* combiner that switches mid-stream reveals a penalty specific to irrevocable problems: with perfect advice, eager switching scores 0.927 — *below both* the pure follower (1.000) and the pure baseline (0.958; `tests/test_combiner_small.py`) — because switching from Ranking to advice-following (*mimic*) mid-run lands the committed matching in an *incompatible hybrid*. In an irrevocable problem the follow/fallback decision must be made *before* the bulk of the commitments, which is why Choo/BEM test a prefix and then *commit* rather than switching dynamically. The dynamic combiner that is cheap insurance for caching does not port cleanly to matching.

6.5 Chapter summary

Test-and-fallback delivers the adaptive robustness envelope of §6.1, but its accept/reject decision is fundamentally limited on strong-baseline inputs: no empirical-ℓ_1 threshold both captures the upside and stays safe (§6.3), and dynamic switching is dominated by test-then-commit (§6.4). The resolution limit of §6.3 is the empirical shadow of the impossibility theorem of Chapter 9.

Chapter 7

External Validity

Chapters 4–6 use synthetic graphs and a synthetic error knob. Is the picture real? We stress it three ways: a genuine, cheap predictor on a real request trace (§7.1); the six real-world graphs (§7.2); and whether *learning* the predictor changes anything (§7.3).

7.1 A real, cheap predictor

We replace the synthetic knob with the cheapest realistic predictor: last-window historical statistics. Real Wikipedia daily pageviews give a live day (the truth) and earlier days (a 1-, 7-, or 30-day-stale forecast), so the error is genuine temporal drift; we map the trace onto a fixed serving topology and consume the forecast through the degree route (MPD) (**Figure 7.1**). Three facts emerge. First, **the cost premise does not bite**: the predictor is a linear-time count (0.108 ms — about 2% of computing OPT once), not an ML inference. Second, **the benefit is real, partial, and never harmful**: a stale forecast captures 27%–68% of the oracle gap (falling with staleness) and always stays above the baseline (0.938–0.957 vs 0.923–0.925, even at 30 days). Third, and why: **topology aggregation makes the cheap predictor order-faithful** — the induced degree predictor's order error is only Kendall-$\tau \approx 0.19$–0.32, roughly *half* the raw histogram's (0.38–0.49) — and since MPD depends only on order (Chapter 5), the aggregated route survives real drift. Consuming the *same* forecast through the raw histogram is catastrophic: blind FollowPrediction collapses to $0.68 \to 0.36$, far below the ≈ 0.92 baseline — exactly what the robust algorithms of Chapters 4 and 6 are for.

7.2 Six real-world graphs

We re-run the degree-prediction roster of Chapter 4 on the six Network-Repository graphs (two Facebook social, two C. elegans biological, two economic input-output), across the same quality columns, with 95% CIs (**Figure 7.2**). The two load-bearing findings are universal. **F1 holds on all six**: naive MPD fed an adversarial predictor falls below the Ranking floor everywhere — by 0.07 (Caltech36) to 0.11 (CE-PG). **F3 is universal and confirms its own logic**: the consistency upside is small everywhere (mean +0.049; range +0.022–+0.077) and smallest exactly where the baseline is strongest — the two dense economic graphs, with Ranking already 0.965/0.977, give the tiniest upsides.

The structural-robustness finding **F2 holds qualitatively on all six** (the augmentations are always less sensitive to prediction quality than naive MPD) and *strictly* on the four social/bio graphs

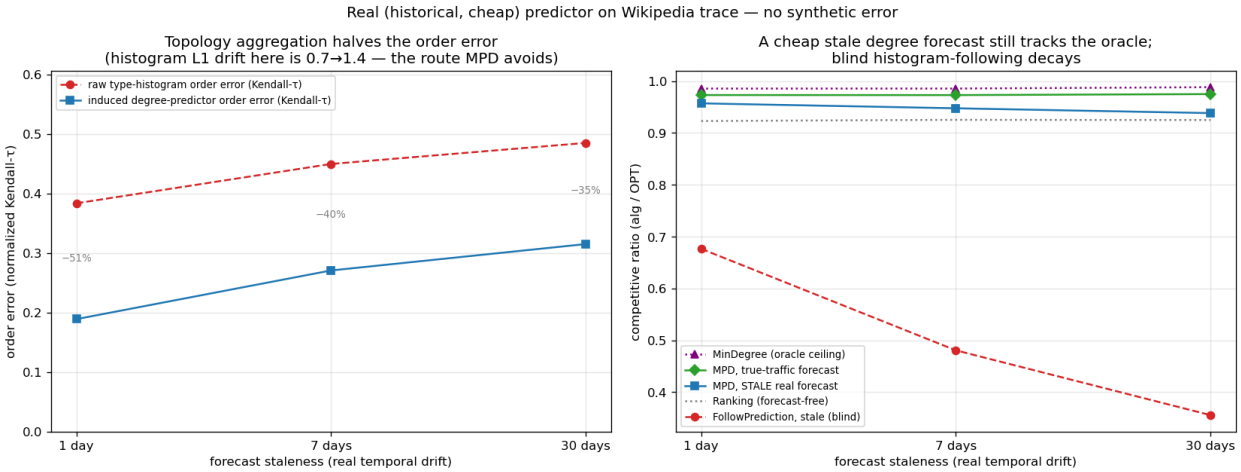

Figure 7.1: A real, cheap predictor (Wikipedia trace). (a) Topology aggregation halves the induced degree predictor's order error relative to the raw histogram. (b) A stale forecast tracks the oracle through the degree route while blind histogram-following decays.

(spread 0.22–0.29× MPD's); on the two dense economic graphs the protection is only *partial* — the augmentation cushions the adversarial drop (0.939 vs naive MPD's 0.893) but cannot clear the unusually high 0.965 floor, dipping ≈ 0.03 below it. This econ boundary is instructive rather than a failure: those graphs are so dense that matching is nearly trivial (Ranking ≈ 0.97, MinDegree = 1.00), so there is neither upside to capture (F3) nor much downside to protect. Finally, F4 is *dramatic*: the worst-case-designed Feldman/Jaillet–Lu are the weakest advice-free entries on the econ graphs (0.73–0.77) and the augmentation lifts them to 0.99 — a +0.26 rescue.

7.3 Does learning the predictor help?

Because MPD consumes the predictor only through order (Chapter 5), one might train it with a rank loss rather than regression. We tested this and report an honest negative. With *deliberately divergent* synthetic features rank-training beats regression sharply (0.989 ≈ oracle vs 0.974, with worse regression error but better order); but the advantage is gated by feature divergence and by graph difficulty (peaking at +1.3%), and, decisively, on real temporal features it *disappears* — rank- and regression-trained predictors produce identical order (Kendall-τ 0.126 vs 0.126) and identical ratio. The dissociation that powers order-aware training is a property of engineered features, not realistic ones. (The fuller account of this exploration is Chapter 8.) The lesson reinforces the thesis: once a predictor is order-faithful — which a cheap historical count already is (§7.1) — neither a better algorithm nor a better-trained predictor buys much on average-case matching.

7.4 Chapter summary

On real predictors, real graphs, and a learned predictor, the same wall stands: the advice-free baseline is near-optimal, unguarded following is unsafe, and predictions buy downside protection rather than performance. Having established the wall empirically across every setting we could reach, the thesis next proves it is necessary (Chapter 9).

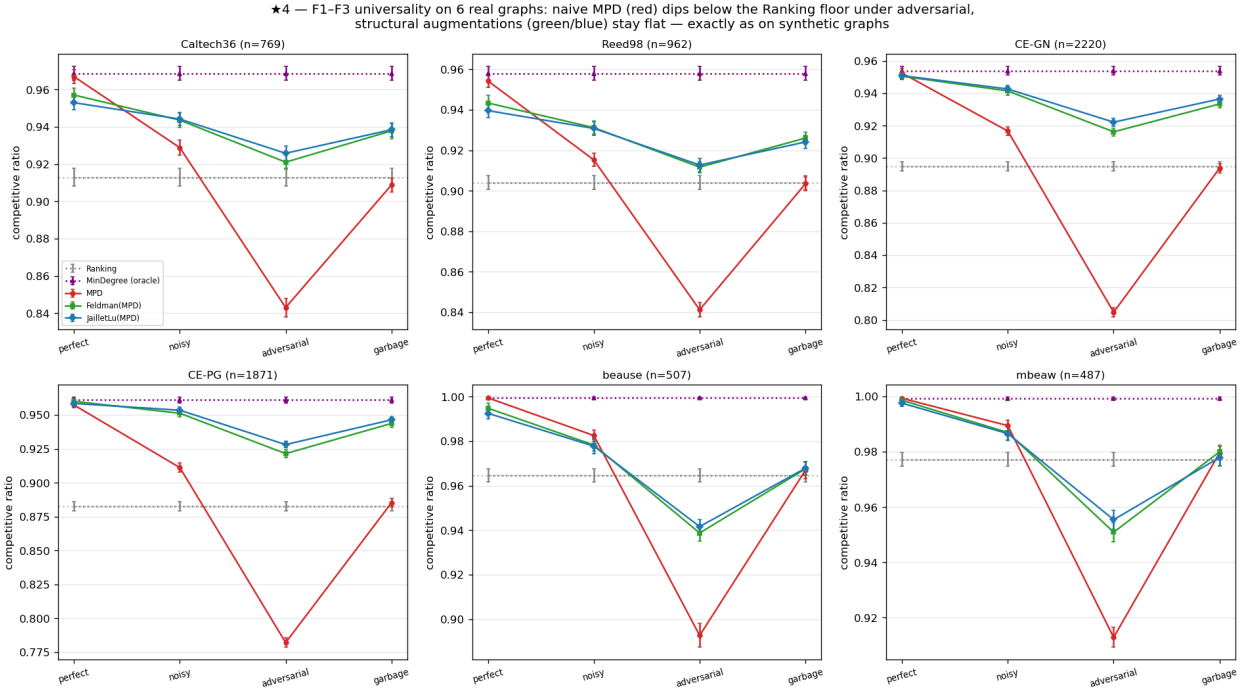

Figure 7.2: F1–F3 on six real graphs. Naive MPD (red) dips below the Ranking floor under adversarial predictions on every graph; the structural augmentations (green/blue) stay flat — as on synthetic graphs.

Chapter 8

Exploratory Directions and Negative Results

The experimental chapters (4–7) establish a wall: on average-case matching the advice-free baseline is near-optimal, so predictions are robustness insurance rather than a performance lever. Before proving this wall is necessary (Chapter 9), we pursued three directions that each *tried to get past it* — learning the predictor to squeeze out more performance (§8.1), finding a serving regime where predictions genuinely help (§8.2), and letting a systematic literature review point us to the highest-value contribution (§8.3). All three returned the same answer, and their convergence is what motivated the theorem. This chapter reports them honestly, including the negatives, because they are part of the evidence and because ruling out ambitious alternatives is itself a result.

8.1 Learning to rank the predictor (a negative result)

Chapter 5 shows that MPD consumes its predictor only through the *order* it induces. This suggests a concrete way to do better: rather than training a predictor to minimize its *regression* error to the true degrees (the standard "predict-then-optimize" objective), train it with an *order-aware* loss — a pairwise rank loss — so it optimizes the quantity the algorithm actually uses. We investigated this in three steps.

M0 — the mechanism exists. With deliberately *divergent* synthetic features (one feature carrying magnitude, another carrying order), a rank-trained linear predictor sharply beats a regression-trained one on the decision metric: matching ratio 0.989 (essentially the oracle) versus 0.974, while the rank-trained predictor has *worse* regression error to the truth (MSE 87.6 vs 33.4) but better order (Kendall-τ 0.058 vs 0.255). The dissociation is real: the worse *fit* gives the better *decision*, confirming that regression is the wrong training objective when it matters.

M1 — but the advantage is doubly gated, and small. Sweeping the feature divergence and the graph difficulty, the rank-advantage is zero when features do not induce an order/magnitude conflict, and zero on easy instances where the baseline is already optimal (the wall, again). It peaks at only +1.3% of the ratio on synthetic graphs; a more favorable framing is *gap-capture*, where rank-training recovers essentially the full oracle gap while regression leaves about 30% of it unrealized — but the gap itself is small.

M3 — and it disappears on real features. The decisive test uses genuine temporal features

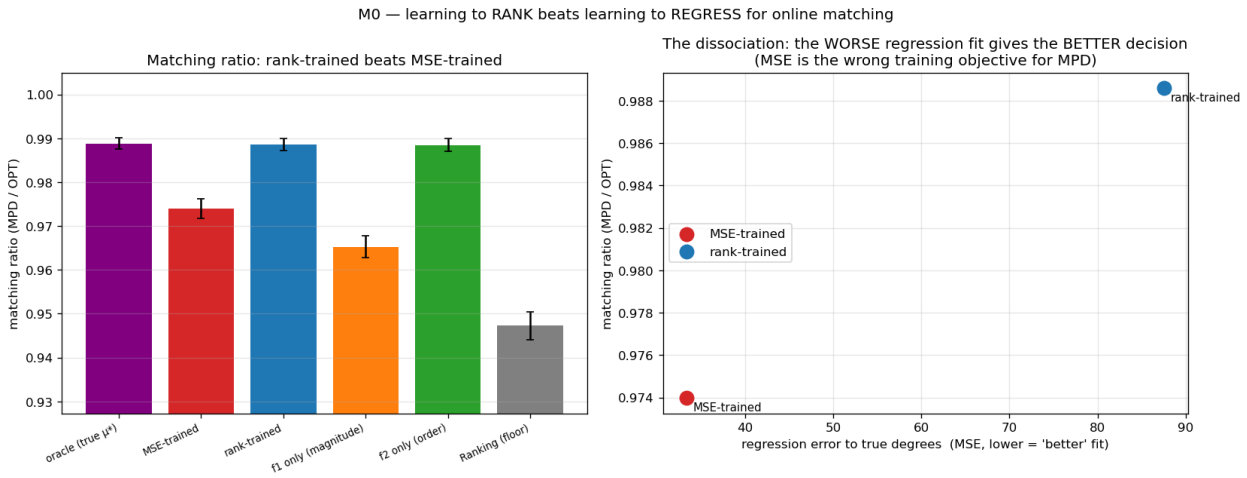

Figure 8.1: M0: with divergent features, rank-training beats regression on the matching ratio (left) *despite* a worse regression fit (right) — the dissociation that shows regression is the wrong training objective for MPD.

from real serving traces (per-resource reference counts over the previous windows) to predict the next window. Here the rank- and regression-trained predictors produce *identical* order (Kendall-τ 0.126 vs 0.126) and identical matching ratio, across every topology and lag configuration tried. The order/magnitude divergence that powers rank-training is a property of *engineered* features; realistic lagged-count features are co-linear noisy estimates of the same popularity, so regression already recovers the order as well as ranking does.

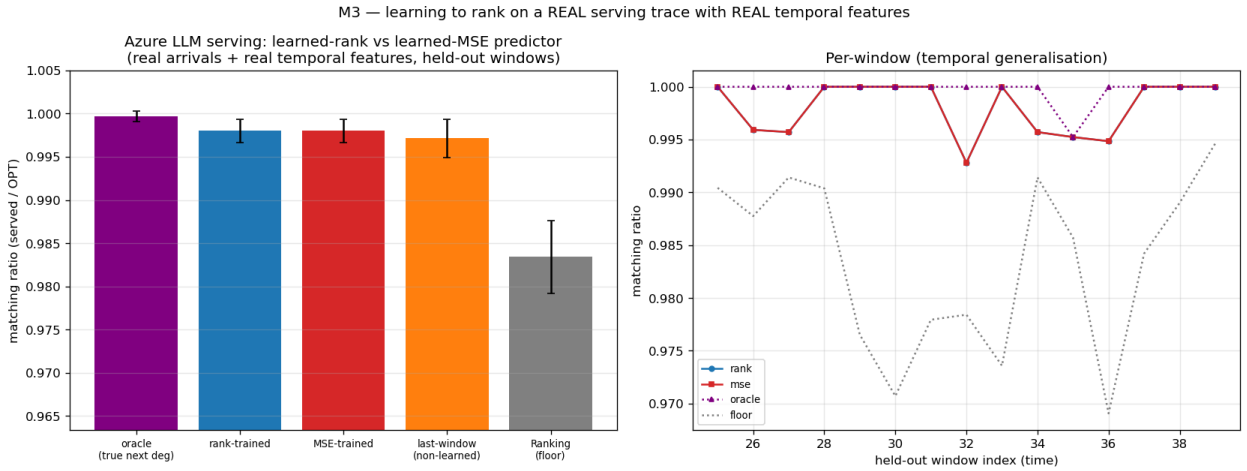

Figure 8.2: M3 — the decisive negative on a real trace (Azure LLM): with real arrivals and real temporal features, the rank-trained and MSE-trained predictors are indistinguishable from each other and from the non-learned last-window count, all near the oracle; the engineered order/magnitude divergence that powers rank-training does not arise.

Verdict. Learning the predictor with a decision-aligned loss does not elevate to a standalone contribution: the win requires a feature divergence that does not arise in practice, and even where it does the payoff is bounded by the (small) baseline-to-oracle gap. The result is folded into the

thesis as a negative that *reinforces* the central finding — once a predictor is order-faithful, which a cheap historical count already is (Chapter 7), neither a better algorithm nor a better-trained predictor buys much on average-case matching.

8.2 Rescuing the serving application with a with-predictions result (a negative probe)

The serving case study (Chapter 10) recovers established systems results and is therefore presented as a case study rather than a novelty claim. We asked whether a *new* actionable with-predictions result could rescue it. The obstacle is again the wall: every serving variant we tried optimizes *throughput* (goodput), and throughput is forgiving — under overload a reactive router fills capacity just as the optimum does, so predictions add nothing. The escape, if one exists, must be a different *objective* on which the reactive baseline is genuinely far from optimal.

We probed the most promising candidate: an **SLO / tail objective** — protecting a tight-SLO class of requests from being dropped — under bursty, non-stationary load, exactly the regime where a reactive policy, lacking foresight, might fail. Using an event-driven simulator we compared non-predictive policies (static capacity reservation; a reactive-adaptive policy that reserves based on *observed* recent load) against a **clairvoyant oracle** that reserves based on the *actual future* burst. Across every regime swept — overload level, uniform vs bursty tight-SLO demand — the best non-predictive policy matches the clairvoyant oracle to within $\leq 3\%$; in the moderate regime a trivial static reservation of one slot drives tight-SLO violations to near zero and *beats* the clairvoyant oracle outright. Two reasons, both robust to the sweep: protecting a tight-SLO minority needs only a small static headroom, no forecast; and bursts are persistent enough that reacting to observed load is almost as good as forecasting it.

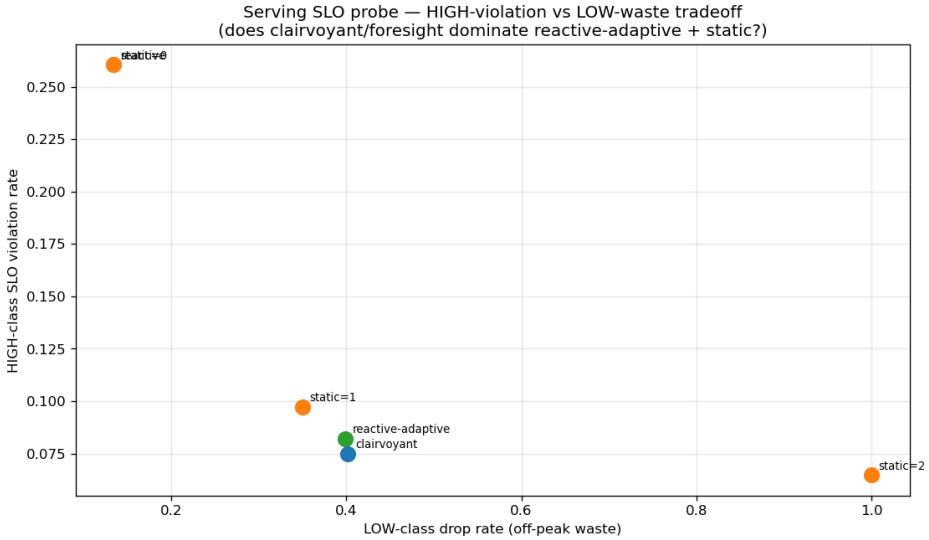

Figure 8.3: Serving SLO probe: on a tight-SLO/tail objective under bursty load, a non-predictive policy (static headroom or reactive-adaptive) matches the clairvoyant oracle to within a few percent — foresight does not help.

Verdict. The tail objective is forgiving too — a third face of the wall, after throughput (Chapters 4–7) and predictor-learning (§8.1). We found no natural regime where foresight helps, so serving

remains a case study. A regime that would break the wall (a non-stationary or adversarial objective where the baseline is far from optimal) is exactly the kind of setting the thesis brackets as future work.

8.3 A literature review that redirected the work

Deciding *which* of our findings were genuinely novel — and worth developing — required a systematic prior-art review rather than intuition. We conducted a large multi-source, adversarially-verified literature search (documented in `docs/LITERATURE_REVIEW.md`) that returned honest, sometimes deflating, verdicts. It confirmed that the unified experimental benchmark and the empirical study of test-and-fallback are unoccupied and worth leading with; that the order-error finding is *partially* pre-empted by ACI's Appendix D and must be reframed as a tightness/measure characterization rather than a discovery (Chapter 5); and that the serving results are largely re-derivations of established systems facts, warranting their demotion to a case study (§8.2, Chapter 10). A second, focused prior-art pass on the theory confirmed that the impossibility of Chapter 9 is unoccupied, while flagging the one risk to defend against (Choo et al.'s constructive baseline-coupling), which shaped the information-theoretic framing of the proof.

This review is itself part of the research process reported here: it turned an undifferentiated pile of findings into a prioritized contribution, redirected effort away from low-value elaborations (weighted-value variants, more baseline comparisons) and toward the theorem, and enforced the honesty guardrails carried throughout the thesis.

8.4 Synthesis: the negatives motivate the theorem

The three explorations point the same way. A better-trained predictor does not help on real features (§8.1); a with-predictions lens does not rescue serving even on a tail objective (§8.2); and the literature review confirmed there was no easy performance win to be had (§8.3). Together with the throughput wall of Chapters 4–7, they show the phenomenon is not confined to one objective, one algorithm, or one dataset — it recurs everywhere we pushed. That robustness of the empirical finding is precisely what suggested it might be *forced*, and it is what motivated the impossibility theorem of Chapter 9: having failed to get past the wall from several directions, we set out to prove that no algorithm of the relevant class can.

Chapter 9

The Impossibility Theorem: Why the Wall Is Necessary

Every experimental chapter met the same wall: on average-case matching the advice-free baseline is near-optimal, predictions buy robustness insurance rather than performance, and — for the test-and-fallback algorithms — no acceptance threshold both captures the tiny upside and stays safe (Chapter 6). This chapter argues the wall is not an accident of a particular threshold, graph, or algorithm, but a *theorem*. To keep the exposition focused we give the intuition and the architecture of the proof here, and defer the full formal development to Appendix B.

9.1 The claim

Informally:

> On strong-baseline instances, no test-and-fallback algorithm whose test inspects only a **sublinear** prefix of the arrivals can be both consistent and robust.

A *test-and-fallback* algorithm (Chapter 3) observes the first k arrivals, decides — by *any* rule — whether to follow the advice or fall back to Ranking, and commits. The claim is that when the budget k is sublinear ($k = o(n)$) and the baseline is strong, the achievable (consistency, robustness) region collapses to the single point "just play the baseline": one can be safe, or capture the upside, but not both.

Figure 9.1 is the empirical face of this. As the baseline weakens (moving right to left), the *potential* upside — how much perfect advice beats the baseline — grows; but the upside a sublinear test can *safely capture* stays pinned near zero. The gap between the two curves is the impossibility.

9.2 The intuition: testability and baseline strength are the same knob

The heart of the argument is a coupling that is easy to state and, once seen, hard to un-see. To decide safely whether to trust the advice, the algorithm must, from its short prefix, tell *good* advice (close enough to the truth to help) from *bad* advice (far enough to hurt). This is a statistical test on the distribution of request types. Such a test is only feasible when there are **few types,**

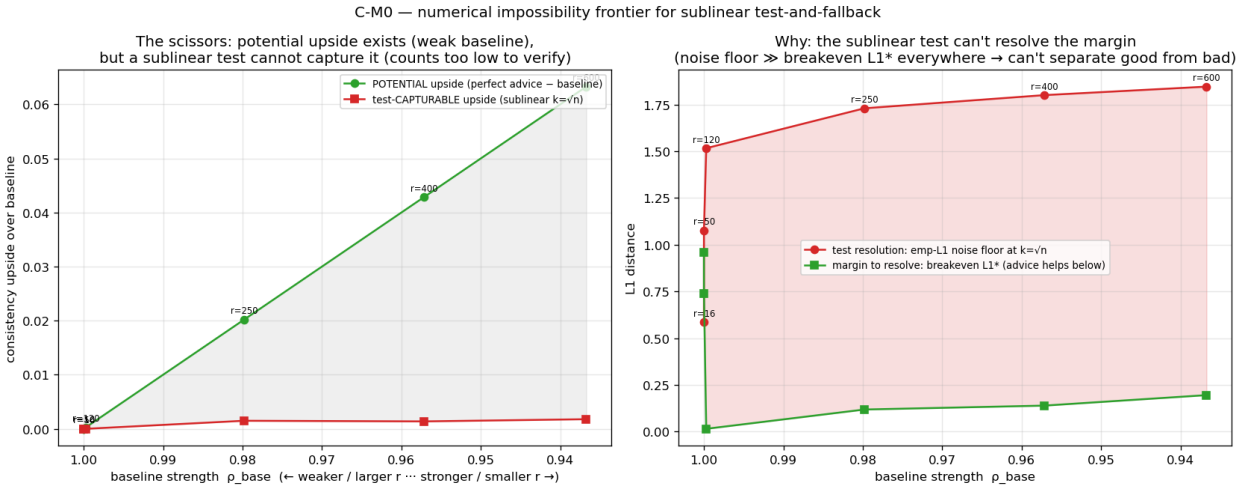

Figure 9.1: The impossibility, numerically (the "scissors"). Left: the *potential* upside grows as the baseline weakens, but the upside a sublinear test can *safely capture* stays near zero. Right: the test's empirical-ℓ_1 resolution sits far above the break-even margin wherever an upside exists.

each arriving many times — otherwise the prefix is too sparse to estimate anything. But few high-count types is *exactly* the regime in which the matching is near-perfect and the advice-free baseline is already near-optimal, so there is nothing worth testing for. Where the baseline is weak enough that advice could genuinely help, the types are many and each is seen only a handful of times, and no sublinear prefix can run the test.

In one sentence: **the structure that makes the test feasible is the structure that makes the baseline near-optimal — so wherever you can test the advice, you do not need it.** The rest of the chapter turns this intuition into a proof.

9.3 The architecture of the proof

The proof rests on three pillars; we state each in words and give its one key formula, leaving the formal details to Appendix B.

Pillar 1 — a trade-off inequality (rigorous). If two instances that require *opposite* decisions (follow vs fall back) look statistically the same over the prefix — that is, their prefix distributions have small total-variation distance γ_k — then no rule on the prefix can serve both. Formally, writing η_c for the fraction of the upside an algorithm forgoes and η_r for its robustness loss, one shows

$$(1-\eta_c) \;\le\; \eta_r + \gamma_k + o(1).$$

When the prefix is uninformative ($\gamma_k \to 0$) the algorithm cannot be both consistent ($\eta_c \to 0$) and robust ($\eta_r \to 0$). This lemma is short and self-contained; it is the two-sided core of the result.

Pillar 2 — a reduction to distribution testing (any rule). The prefix is *literally* a set of i.i.d. samples from the type distribution. We build a family in which following the advice helps exactly when the advice is ℓ_1-close to the truth and hurts exactly when it is ℓ_1-far; on it, a consistent-and-robust algorithm would in effect *distinguish close from far from the samples* — a **tolerant** distribution-testing problem. Because the algorithm's decision is an arbitrary function of

the samples, the lower bound below rules out *every* rule, not just the empirical-distance threshold used in practice — which is what distinguishes our result from the constructive baseline-coupling already present in Choo et al.'s algorithm.

Pillar 3 — tolerant testing is nearly as hard as learning the distribution. The final ingredient is a known and, at first, surprising fact from distribution testing. *Verifying* that a distribution equals a known one needs only $\Theta(\sqrt{r})$ samples — sublinear in the number of types r. But *tolerant* testing — distinguishing "somewhat close" from "somewhat far," which is what safe advice-use requires — needs $\tilde{\Theta}(r/\log r)$ samples, nearly *linear* in r [17, 18]. Our construction has $r = \Theta(n)$ types, so the test needs $\tilde{\Theta}(n/\log n)$ samples — more than any sublinear prefix provides. The good-advice side is genuinely a *ball* around the truth (not a single point), which is what places the problem in the hard tolerant regime rather than the easy $\sqrt{r}$ one. A side benefit of the construction is that the map from advice error to matching loss is an *exact linear law* (verified numerically), which pins the constants cleanly.

Chaining the three: on the family, any $(1-o(1))$-consistent, robust, sublinear-test algorithm would solve a tolerant test it provably cannot, so no such algorithm exists.

9.4 The theorem

> **Theorem (informal).** For every sublinear test budget $k = o(n/\log n)$ there is a known-i.i.d. matching family with $\Theta(n)$ types and a strong baseline on which no test-and-fallback algorithm — by any rule on its prefix — is simultaneously $(1 - o(1))$-consistent and robust. The achievable (robustness, consistency) region collapses to the baseline point.

Together with the easy strong-baseline side (few types ⇒ near-optimal baseline ⇒ no upside to capture), this is precisely the scissors of Figure 9.1, now with a proof: the upside lives only where the tolerant test is infeasible.

9.5 Scope, status, and honesty

Scope. The strong form is stated for the regime with $\Theta(n)$ types — which is the only regime with an upside to contest, and hence the natural one; a version that also bites at constant r is left open. The reduction inherits the empirical-ℓ_1 modeling of the test used by the algorithms it bounds (Chapter 3).

Relation to prior work. Choo et al. and BEM give algorithms (upper bounds) and no lower bound in this model; Choo et al.'s threshold already couples to the baseline ratio, but *constructively*. Our contribution is the two-sided, *rule-independent* impossibility and the identification of testability with baseline strength; the reduction from distribution-testing sample complexity to the value of advice in an online algorithm is, to our knowledge, new.

Status (stated plainly). The trade-off inequality (Pillar 1) is proved; the construction's constants and the linear conversion law are derived and numerically verified; the tolerant-testing barrier (Pillar 3) is a cited, exact result on the theorem-preserving side of the tolerant/non-tolerant divide. One routine step of the construction — exhibiting the two indistinguishable witness distributions guaranteed by the testing lower bound — and a final review remain before the full proof is complete.

The thesis presents the theorem with this status stated, as befits in-progress theoretical work; the empirical scissors of Figure 9.1 is independent evidence that the statement is true.

Chapter 10

Application Case Study: AI-Inference Serving

The thesis's abstraction is not confined to classical matching markets; it instantiates a contemporary systems problem. This chapter casts request routing in an AI-inference serving system as online matching and shows the framework recovers the field's established results. It is presented, deliberately, as a **case study, not a novelty claim** — the systems facts below are known, and the with-predictions vocabulary is a re-labeling of them (a decision justified by the literature review of Chapter 8 and the negative probe summarized in §10.2).

10.1 The serving instantiation

We map serving to online b-matching: the offline resources are model replicas or cache shards, each with a **capacity** c (it can serve up to c concurrent requests, hence b-matching rather than 1-matching); arrivals are requests drawn from a non-uniform, bursty traffic distribution over request types; an edge is a capability or cache affinity; and **goodput** — the fraction of requests served — is the competitive ratio against the b-matching optimum. We instantiate this on three real traces (Wikipedia pageviews, an Azure LLM inference trace, and the Mooncake prefix-cache trace [20]) and across four serving concerns.

- **Capacity as robustness** (**Figure 10.1**). Increasing the capacity c smooths the effect of a bad prediction: a capacity-aware baseline stays safe, while *blindly* trusting a traffic forecast degrades *further* as capacity grows. Capacity is thus a *substitute* for algorithmic robustness — the systems analogue of the robustness-insurance thesis.
- **Forecasts vs live load** (**Figure 10.2**). Under dynamic service times (requests hold a slot for a real duration, released event-by-event), a live-load signal beats a stale traffic forecast — a reactive load balancer outperforms a forecast-following router when the forecast has aged.
- **Cache-affinity routing** (**Figure 10.3**). For prefix-cache-aware routing, a *stable* affinity router beats a reactive one — the reverse of the traffic-forecast case, because cache locality rewards persistence [21].

Each of these recovers an established systems result cleanly, demonstrating that the framework instantiates the problem faithfully.

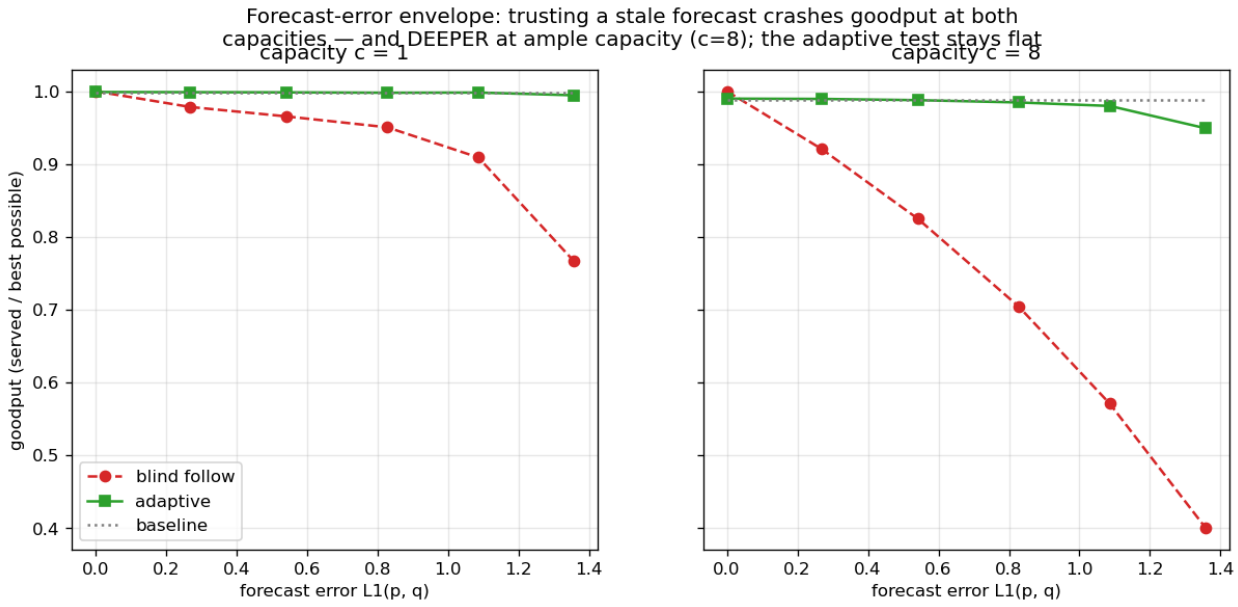

Figure 10.1: Capacity as robustness: as forecast error grows, blindly following the forecast crashes goodput at both capacities — and *deeper* at ample capacity ($c = 8$) — while the adaptive test stays flat at the capacity-aware baseline.

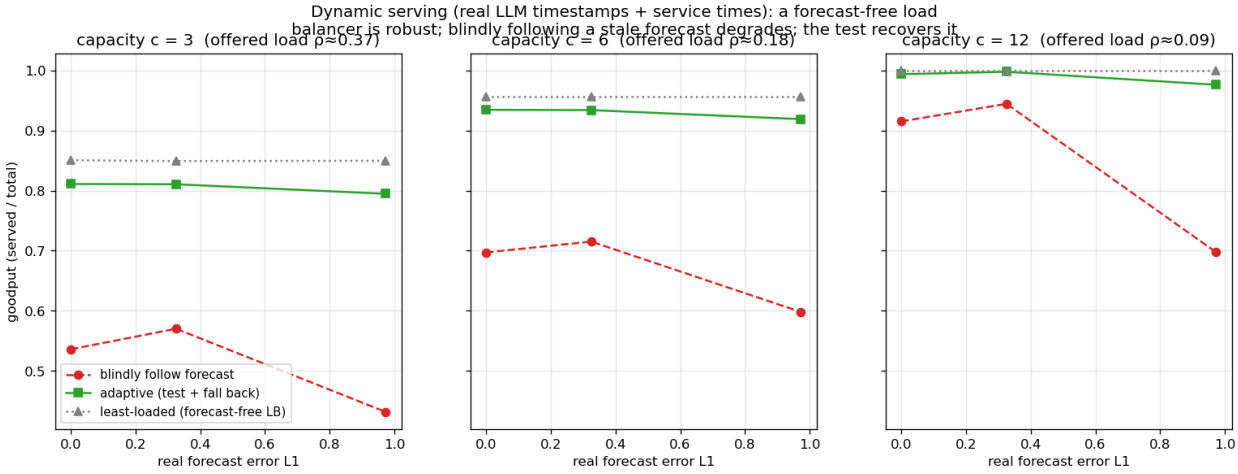

Figure 10.2: Forecasts vs live load (real LLM timestamps and service times): a forecast-free least-loaded balancer is robust at every capacity, while blindly following a stale forecast degrades with its error; an adaptive test recovers most of the gap.

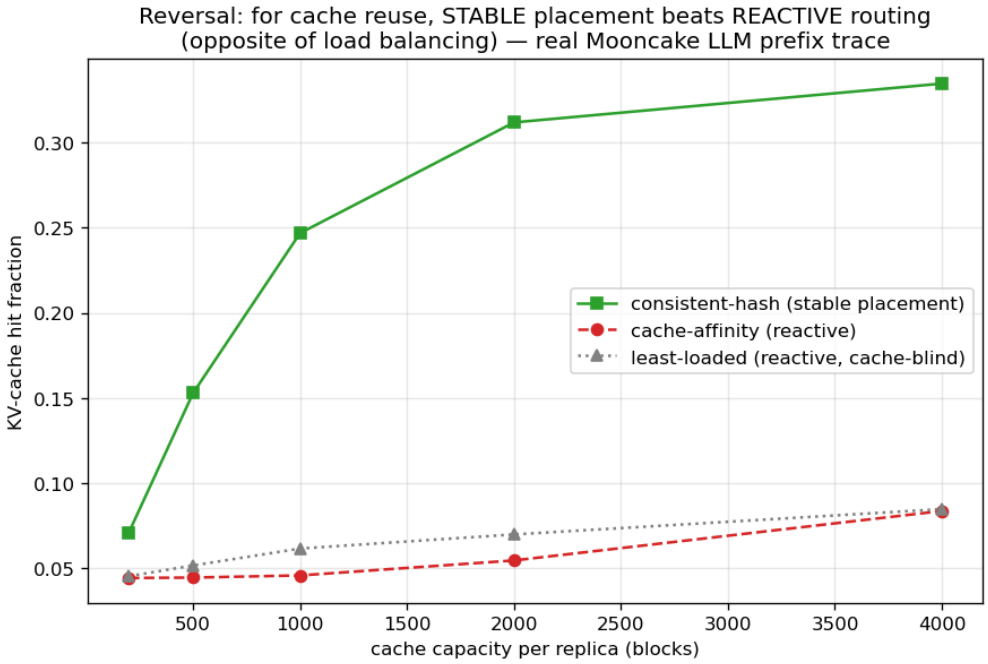

Figure 10.3: The cache-affinity reversal (real Mooncake prefix trace): for KV-cache reuse, a *stable* consistent-hash placement beats reactive routing — the opposite of the load-balancing case — because cache locality rewards persistence.

10.2 A probe for a genuinely new result, and its negative

Because the literature suggested the with-predictions lens might yield a *new* actionable serving result on a tail objective, we probed it (the full account is in Chapter 8). On an SLO/tail objective — protecting a tight-SLO class of requests under bursty load — a non-predictive policy (static headroom, or a reactive-adaptive reservation based on observed load) matches a clairvoyant oracle to within $\leq 3\%$ across every regime swept; a trivial static reservation of one slot even beats the oracle in the moderate regime. Two reasons, both robust: protecting a tight-SLO minority needs only a small static headroom, no forecast; and bursts are persistent enough that reacting to observed load is nearly as good as forecasting it. The tail objective is forgiving too — a third face of the thesis's wall, after throughput (Chapters 4–7) and predictor-learning (Chapter 8). We found no natural regime where foresight helps, so serving remains a case study.

10.3 Chapter summary

The serving instantiation shows the framework reaches a live systems problem and recovers its established results — capacity as a robustness substitute, live load over stale forecasts, stability for cache locality — and that even a tail objective does not open a genuine with-predictions win. The application is a faithful case study of the thesis, not an independent contribution.

Chapter 11

Conclusion and Future Work

11.1 Summary

This thesis studied learning-augmented algorithms for online bipartite matching by first building a common experimental foundation and then explaining what it revealed. On a single harness — validated against the published Borodin et al. study (Chapter 3) — we compared the advice-free baselines, MinPredictedDegree and its augmentations, and the test-and-fallback algorithms, across synthetic graphs, six real-world graphs, and real request traces (Chapters 4–7). Across every setting the same picture appeared: the advice-free baseline is already near-optimal on average-case inputs, so the *consistency* upside of a good prediction is small; unguarded prediction-following crashes *below* the baseline under bad predictions; and the value of the sophisticated algorithms is downside protection, delivered by either a structural or an adaptive robustness mechanism with distinct trade-offs. We sharpened the sub-questions — that the (small) loss is governed by a Kendall-τ order error rather than magnitude (Chapter 5, crediting the prior order-dependence result of ACI), and that the adaptive test has a threshold-calibration pathology and a resolution limit (Chapter 6) — and we honestly reported the directions that did *not* pan out: learning the predictor for extra performance, and finding a serving regime where predictions genuinely help (Chapter 8).

Finally, we argued the recurring wall is *necessary* (Chapter 9): no test-and-fallback algorithm whose test inspects only a sublinear prefix can be both consistent and robust on strong-baseline matching, because the structure that makes the prefix distribution-test feasible is the structure that makes the baseline near-optimal. Experiments discover the wall; theory explains why it must be there. The single sentence that unifies the thesis is: **on average-case online matching, predictions are robustness insurance rather than a performance lever — and where you can test the advice, you do not need it.**

11.2 Limitations

- **Input model.** We work in the known-i.i.d. model. Because Known-I.I.D. $\leq$ Random-Order in difficulty, the algorithms' guarantees carry over, but the empirical wall is an *average-case* statement; we do not claim it for adversarial arrival order.
- **Test model.** Following the original authors, the test-and-fallback experiments and the theorem's reduction use an empirical-ℓ_1 surrogate for the (unimplemented) distribution tester.
- **Prediction-object heterogeneity.** The degree- and histogram-prediction families do not

map onto every graph, which is why they are reported in parallel panels rather than one table.
- **Data breadth.** Each real modality is exercised by one trace.
- **Theory scope and status.** The impossibility theorem is strong-form in the $r = \Theta(n)$ regime — the regime with an upside to contest — and one routine step of its proof (the witness-instance construction) plus a final review remain before it is fully typeset; a version biting at constant r is open.

11.3 Future work

The thesis brackets, rather than resolves, the settings in which predictions might genuinely help online matching, and these are the natural next directions.

- **Beyond average-case inputs.** The wall is an average-case phenomenon. Adversarial or non-stationary arrival orders, where the advice-free baseline is provably far from optimal, are where predictions should carry real value — and where a *positive* counterpart to this thesis's impossibility might be proved.
- **Beyond throughput.** Objectives on which the baseline is not near-optimal — tail latency, per-type fairness, migration or recompute cost — may admit genuine with-predictions gains that the goodput objective forecloses; our serving SLO probe (Chapter 8) closed the simplest such attempt but not the space.
- **Completing and strengthening the theory.** Closing the remaining construction step and the review; extending the impossibility to constant r or to other prediction objects; and a matching *upper* bound — for instance, showing that a recalibrated-threshold algorithm is optimal among test-and-fallback schemes — would turn Chapter 9 into a tight characterization.
- **Better tests, honestly.** Whether a super-linear or amortized test, or a test that reuses online decisions as samples, can escape the tolerant-testing barrier is an open and practically motivated question.

11.4 Closing

Predictions are a powerful tool for online algorithms, but this thesis is a study of their *limits* on one well-understood problem. On average-case online matching, the honest verdict is that a cheap, order-faithful predictor already captures nearly all there is to capture, that the sophisticated machinery earns its keep as insurance rather than as performance, and that no sublinear test can do better — because the very feasibility of the test is a signal that the prediction was not needed. Recognizing where predictions cannot help is, we hope, as useful as knowing where they can.

References

[1] R. M. Karp, U. V. Vazirani, and V. V. Vazirani. An optimal algorithm for on-line bipartite matching. *STOC*, 1990.

[2] J. Feldman, A. Mehta, V. Mirrokni, and S. Muthukrishnan. Online stochastic matching: Beating 1-1/e. *FOCS*, 2009.

[3] V. H. Manshadi, S. Oveis Gharan, and A. Saberi. Online stochastic matching: Online actions based on offline statistics. *Mathematics of Operations Research*, 37 (4), pp. 559–573, 2012.

[4] P. Jaillet and X. Lu. Online stochastic matching: New algorithms with better bounds. *Mathematics of Operations Research*, 2014.

[5] A. Borodin, C. Karavasilis, and D. Pankratov. An experimental study of algorithms for online bipartite matching. *arXiv preprint arXiv:1808.04863*, 2018.

[6] M. Mitzenmacher and S. Vassilvitskii. Algorithms with predictions. *Beyond the worst-case analysis of algorithms*, Cambridge University Press, pp. 646–662, 2020.

[7] T. Lykouris and S. Vassilvitskii. Competitive caching with machine learned advice. *ICML*, 2018.

[8] A. Wei and F. Zhang. Optimal robustness-consistency trade-offs for learning-augmented online algorithms. *NeurIPS*, 2020.

[9] J. Chłędowski, A. Polak, B. Szabucki, and K. Żołna. Robust learning-augmented caching: An experimental study. *ICML*, 2021.

[10] …. Yoshinaga and Y. Kawase. Analyzing the effect of prediction accuracy on the distributionally-robust competitive ratio. *arXiv preprint arXiv:2601.06813*, 2026.

[11] A. Aamand, J. Y. Chen, and P. Indyk. (Optimal) online bipartite matching with degree information. *NeurIPS*, 2022.

[12] D. Choo, T. Gupta, and others. Online bipartite matching with imperfect advice. *ICML*, 2024.

[13] J. Jiao, Y. Han, and T. Weissman. Minimax estimation of the L_1 distance. *IEEE Transactions on Information Theory*, 64 (10), pp. 6672–6706, 2018.

[14] …. Burathep, T. Erlebach, W. K. Moses, and others. Learning-augmented online bipartite matching in the random arrival order model. *SOFSEM*, 2026.

[15] L. Paninski. A coincidence-based test for uniformity given very sparsely sampled discrete data. *IEEE Transactions on Information Theory*, 54 (10), pp. 4750–4755, 2008.

[16] G. Valiant and P. Valiant. An automatic inequality prover and instance optimal identity testing. *SIAM Journal on Computing*, 46 (1), pp. 429–455, 2017.

[17] G. Valiant and P. Valiant. Estimating the unseen: An $n/\log n$-sample estimator for entropy and support size, shown optimal via new CLTs. *STOC*, 2011.

[18] C. L. Canonne, A. Jain, G. Kamath, and J. Li. The price of tolerance in distribution testing. *COLT*, 2022.

[19] J. E. Hopcroft and R. M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 1973.

[20] others. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *arXiv preprint arXiv:2407.00079*, 2024.

[21] others. Preble: Efficient distributed prompt scheduling for LLM serving. *arXiv preprint arXiv:2407.00023*, 2024.

Appendix A

Reproduction Guide

Every quantitative result in this thesis is regenerated from a fixed seed by a single script. This appendix maps each figure and table to its script, gives the commands and runtimes, lists the full Phase-2 reproduction tables deferred from Chapter 3, and records data provenance.

A.1 Environment and principles

- **Stack:** Python 3.12; NumPy 1.26+, SciPy 1.13, NetworkX 3.3, Matplotlib (plots only).
- **Determinism:** all randomness derives from one master seed (default `0`) via NumPy's `default_rng(seed).spawn(k)`, which yields independent, reproducible streams (graph, instance, algorithm randomness, prediction perturbation). Results are bit-stable across NumPy minor versions from 1.25 (BitGenerator-based spawning).
- **Paired trials:** within a comparison, every algorithm and error level reuses the same graph, arrival sequence, `OPT`, and tie-break seed, so differences are attributable to the prediction alone.
- **Confidence intervals:** 95% normal-approximation half-widths over trials.
- Each script writes `results/<name>.json` (raw means/CIs) and `results/<name>.png` (plot), and prints per-step progress.

A.2 Figure / table → script map

Thesis object	Script	Output	≈ runtime
Fig 3.1 (ER U-curve)	`scripts/run_er_full.py`	`results/er_full.{json,png}`	~20 min
Fig 3.2 (left-regular)	`scripts/run_left_regular.py`	`results/left_regular.{json,png}`	~10 min
Table 4.1, Fig 4.1 (unified benchmark)	`scripts/run_unified_benchmark.py`	`results/unified_benchmark.{json,png}`, `unified_benchmark_tables.md`	~100 s
Fig 4.2 (consistency–robustness plane)	`scripts/run_consistency_robustness.py`	`results/consistency_robustness.{json,png}`	~1 h
Fig 5.1 (order-error vs ACI)	`scripts/run_order_vs_theory.py`	`results/order_vs_theory.{json,png}`	~30 s
Fig 6.1 (envelope), Fig 6.2 (prefix sweep)	`scripts/run_choo_bem.py`	`results/choo_bem_{envelope,prefix}.png`	~20 min

Thesis object	Script	Output	≈ runtime
Fig 6.3, recalibration (§6.3)	`scripts/run_recalibration_ensemble.py`	`results/recalibration_*.png`	~1.5 min
Fig 7.1 (real predictor)	`scripts/run_real_predictor.py`	`results/real_predictor.{json,png}`	~15 s
Fig 7.2 (six real graphs)	`scripts/run_realworld_robustness.py`	`results/realworld_robustness.{json,png}`	~65 s
Fig 8.1 (M0 rank vs MSE)	`scripts/run_rank_vs_mse_mve.py`	`results/rank_vs_mse_mve.{json,png}`	~10 s
M1 sweep (§8.1, no figure)	`scripts/run_rank_when_it_matters.py`	`results/rank_when_it_matters.{json,png}`	~20 s
Fig 8.2 (M3 real-trace learning)	`scripts/run_rank_real_trace.py`	`results/rank_real_trace.{json,png}`	~10 s
Fig 8.3 (serving SLO probe)	`scripts/run_serving_slo_probe.py`	`results/serving_slo_probe.{json,png}`	~1 s
Fig 9.1 (impossibility frontier)	`scripts/run_impossibility_frontier.py`	`results/impossibility_frontier.{json,png}`	~6 s
Figs 10.1–10.3, serving (Ch 10)	`scripts/run_serving.py`, `run_serving_trace.py`, `run_serving_dynamic.py`, `run_prefix_cache.py`	`results/serving_*.png`, `prefix_cache_*.png`	varies
Real-world Borodin Tables 3/4 (validation)	`scripts/run_realworld.py`	`results/realworld.json`	a few min

A.3 Reproduction commands

```
# from the project root (matching-experiments/)
# correctness anchors - 7 hand-verifiable test files, all pass:
for t in tests/test_*.py; do python3 "$t"; done

# regenerate the headline results (fast ones):
python3 scripts/run_unified_benchmark.py        # Table 4.1, Fig 4.1
python3 scripts/run_order_vs_theory.py          # Fig 5.1
python3 scripts/run_real_predictor.py           # Fig 7.1
python3 scripts/run_realworld_robustness.py     # Fig 7.2
python3 scripts/run_impossibility_frontier.py   # Fig 9.1
python3 scripts/run_rank_vs_mse_mve.py          # Fig 8.1
python3 scripts/run_rank_when_it_matters.py     # M1 (§8.1)
python3 scripts/run_rank_real_trace.py          # Fig 8.2
python3 scripts/run_serving_slo_probe.py        # Fig 8.3
python3 scripts/run_consistency_robustness.py   # Fig 4.2 (replots Table 4.1)

# the long (max-flow / Hopcroft-Karp-bound) sweeps:
python3 scripts/run_er_full.py                  # Fig 3.1  (~20 min)
python3 scripts/run_left_regular.py             # Fig 3.2  (~10 min)
```

```
python3 scripts/run_choo_bem.py                    # Fig 6.1, 6.2 (~20 min)
```

All scripts hardcode seed `0` unless noted; re-running reproduces the reported numbers.

A.4 Full Phase-2 reproduction tables (deferred from §3.6)

Setup: $n = 1000$, $m = n$, 100 trials per parameter value, seed 0. Target is *qualitative* agreement with Borodin et al. (Python/NetworkX vs the paper's C++/Edmonds–Karp; absolute differences ≤ 0.02 accepted).

Erdős–Rényi, competitive ratio at selected c (SG=SimpleGreedy, Rk=Ranking, F/J = Feldman/Jaillet–Lu, -NG/-G = non-greedy/greedy):

c	SG	Rk	F-NG	F-G	J-NG	J-G
0.10	0.9995	0.9994	1.0000	1.0000	0.9991	1.0000
1.90	0.9362	0.9363	0.9033	0.9637	0.9202	0.9602
4.90	**0.8640**	0.8655	0.7648	**0.8845**	0.7949	**0.8854**
8.90	0.9094	0.9088	0.7314	0.9123	0.7662	0.9120
14.90	0.9487	0.9486	0.7290	0.9488	0.7640	0.9483

Per-algorithm minima (ER): SG 0.8640 @ c=4.9; Rk 0.8649 @ 4.7; F-NG 0.7288 @ 13.5; F-G 0.8833 @ 5.3; J-NG 0.7616 @ 14.1; J-G 0.8839 @ 5.3.

Random left-regular, competitive ratio at selected d**:**

d	SG	Rk	F-NG	F-G	J-NG	J-G
1	1.0000	1.0000	0.9845	1.0000	0.9845	1.0000
2	0.9539	0.9537	0.8769	0.9679	0.8945	0.9659
5	**0.8905**	0.8900	0.7595	0.9008	0.7909	0.9022
10	0.9275	0.9278	0.7317	0.9276	0.7677	0.9290
30	0.9770	0.9767	0.7308	0.9757	0.7633	0.9754

Paper-claim checklist (all verified ✓): greedy minima near $c \approx 4.9$ / $d = 5$; Ranking $\approx$ SimpleGreedy (max diff 0.0017 ER, 0.005 LR — the paper omits Ranking's curve for this reason); non-greedy variants degrade monotonically as c, d grow; greedy complex variants $\approx$ SimpleGreedy asymptotically; the c=14.9 non-greedy ordering (J-NG 0.764 > F-NG 0.729) matches the paper's worst-case-bound ordering. A cross-family observation: the non-greedy algorithms converge to the same asymptotic constants in both families (0.729 Feldman, 0.764 Jaillet–Lu, within 0.002), above their worst-case bounds by +0.06 / +0.03 — an early instance of the average case being more benign than the worst case.

A.5 Data provenance

Real data is stored locally under `data/` (large; excluded from version control).

- **Real graphs (Chapter 7, §3.6 validation):** six Network Repository graphs — `socfb-Caltech36`, `socfb-Reed98`, `bio-CE-GN`, `bio-CE-PG`, `econ-beause`, `econ-mbeaflw` — as MatrixMarket `.mtx` / whitespace `.edges`, reduced to simple undirected graphs and converted to bipartite by random balanced partition (Borodin Table 3) or duplicating double-cover (Table 4).
- **Traces (Chapters 7, 8, 10):** Wikipedia "top articles per day" JSON for four days (`data/trace/wiki/`, used as live day vs 1/7/30-day-stale forecasts); the Azure LLM inference trace (`data/trace/azure_llm/`, context/generated token counts with timestamps); the Mooncake conversation trace (`data/trace/mooncake/`, per-request `hash_ids` for prefix-cache blocks).

A.6 Theory verification snippets (Chapter 9)

The single-cell constants of the construction (per-cell OPT, baseline, and the advantage/L1 formulas, §9.3 Pillar-3 / Chapter 9) and the exact affine conversion law $\mathbb{E}[\text{follow-ratio}] = \rho_{\text{perfect}} - \frac{1}{2}\ell_1(p, q)$ were verified numerically to three decimals by short simulations documented in the project notes (`docs/T1_W1_single_cell.md`, `T1_W2_W3a_closeout.md`); e.g. at $\theta = 0.6$, bias $|s - \frac{1}{2}| = 0.3$, the simulated per-cell advantage ± 0.119 matches the formula $\pm\theta|s - \frac{1}{2}| = \pm 0.12$, and the aggregate follow-ratio matches the affine law at every advice level. These are sanity checks on the construction, not part of the formal proof (Appendix B), whose remaining step is noted in §9.5 and §B.7.

Appendix B

Proof Details for the Impossibility Theorem

This appendix records the formal development behind Chapter 9: the model and the algorithm class (§B.1), the master trade-off inequality with its proof (§B.2), the rare-resource cell construction and its closed-form constants (§B.3), the exact affine law converting advice error into matching ratio (§B.4), the reduction to tolerant distribution testing and the sample-complexity barrier (§B.5), the assembled theorem (§B.6), and a plain statement of what remains open (§B.7). The numerical verifications quoted here are reproduced by the snippets referenced in Appendix A.6.

B.1 Setup and the algorithm class

Instances. Known-i.i.d. online bipartite matching (Chapter 3): a type graph over r online types and a set of offline resources; n arrivals drawn i.i.d. from a type distribution p over $[r]$; $\mathrm{OPT}(I)$ is a maximum matching of the realized instance I.

Advice. A type histogram $q \in \Delta([r])$ (the Choo/BEM prediction object). *Following* the advice — the *mimic* policy $\mathrm{Mimic}(q)$ — builds a maximum matching on the advice graph and routes each arrival to its type's advice partner (Chapter 3).

Baseline. The advice-free Ranking algorithm B, with $\mathbb{E}[B]/\mathbb{E}[\mathrm{OPT}] = \rho_{\mathrm{base}}$ on the family under study.

The test-and-fallback class $\mathcal{A}_k$. An algorithm $A \in \mathcal{A}_k$ observes the advice q and the prefix $X_{1:k}$ (the types of the first k arrivals), outputs a — possibly randomized — decision $D \in \{\mathrm{F}, \mathrm{R}\}$ *by any measurable rule*, and then plays $\mathrm{Mimic}(q)$ on arrivals $k+1, \dots, n$ if $D = \mathrm{F}$ and B otherwise. The class covers every decision rule, not only the empirical-ℓ_1 threshold used by the concrete algorithms.

Consistency and robustness. For an instance distribution $\mathcal{D}$ with advice q, write $R(\mathcal{D}) = \mathbb{E}[\mathrm{ALG}]/\mathbb{E}[\mathrm{OPT}]$. Consistency is R under good advice; robustness is R under bad advice (Chapter 3).

Sublinearity (A0). $k = o(n)$. The prefix's own contribution to the matching is at most $k = o(n)$, shifting every ratio by $O(k/n) = o(1)$; we absorb this into the $o(1)$ terms.

B.2 The master trade-off inequality

Lemma B.1 (master trade-off). Let G, Bd be two instance distributions sharing the *same* advice q, with prefix laws $\mathcal{L}_G, \mathcal{L}_{\mathrm{Bd}}$ on $X_{1:k}$, and let $\gamma_k := \mathrm{TV}(\mathcal{L}_G, \mathcal{L}_{\mathrm{Bd}})$. Suppose following q gains δ under G and loses Δ under Bd relative to the baseline:

$$\mathbb{E}_G[\mathrm{Mimic}]/\mathrm{OPT} \geq \rho_{\mathrm{base}} + \delta, \qquad \mathbb{E}_{\mathrm{Bd}}[\mathrm{Mimic}]/\mathrm{OPT} \leq \rho_{\mathrm{base}} - \Delta,$$

while B attains $\rho_{\mathrm{base}} \pm o(1)$ under both. Parameterize an $A \in \mathcal{A}_k$ by η_c, the fraction of the upside it forgoes under G (captured upside $= (1-\eta_c)\,\delta$), and η_r, its robustness loss as a fraction of Δ under Bd (loss $= \eta_r\,\Delta$). Then

$$(1-\eta_c) \;\leq\; \eta_r + \gamma_k + o(1). \tag{$\star$}$$

Proof. Condition on the decision D under G:

$$\mathbb{E}_G[\mathrm{ALG}]/\mathrm{OPT} = P_G(\mathrm{F}) \cdot \frac{\mathbb{E}_G[\mathrm{Mimic}]}{\mathrm{OPT}} + P_G(\mathrm{R}) \cdot \rho_{\mathrm{base}} \pm o(1) \;\geq\; \rho_{\mathrm{base}} + \delta\, P_G(\mathrm{F}) - o(1).$$

The captured upside is $(1-\eta_c)\delta$ by definition, so $P_G(\mathrm{F}) \geq 1 - \eta_c - o(1)$. Symmetrically under Bd, $\mathbb{E}_{\mathrm{Bd}}[\mathrm{ALG}]/\mathrm{OPT} = \rho_{\mathrm{base}} - \Delta\, P_{\mathrm{Bd}}(\mathrm{F}) \pm o(1)$, so the robustness loss is $\Delta\, P_{\mathrm{Bd}}(\mathrm{F}) \pm o(1)$, giving $P_{\mathrm{Bd}}(\mathrm{F}) = \eta_r + o(1)$. Finally, D is a (randomized) function of $(X_{1:k}, q)$ and q is identical across G and Bd, so by the coupling characterization of total variation, $|P_G(\mathrm{F}) - P_{\mathrm{Bd}}(\mathrm{F})| \leq \gamma_k$. Chaining the three displays yields $1 - \eta_c - o(1) \leq P_G(\mathrm{F}) \leq P_{\mathrm{Bd}}(\mathrm{F}) + \gamma_k = \eta_r + \gamma_k + o(1)$, which is $(\star)$. ■

When the prefix is uninformative ($\gamma_k \to 0$), $(\star)$ forbids $\eta_c \to 0$ and $\eta_r \to 0$ simultaneously: the algorithm cannot be both consistent and robust. The remaining work is to construct a family on which $\gamma_k = o(1)$ *coexists* with meaningful stakes $\delta, \Delta = \Theta(1 - \rho_{\mathrm{base}})$.

Remark (why a two-point argument is not enough). For a single pair (G, Bd) whose per-arrival laws differ, the stakes themselves are capped by the distinguishability: $\delta, \Delta = O(\gamma_k)$, so $(\star)$ alone gives only a vacuous bound. To obtain $\delta, \Delta = \Theta(1)$ *while* $\gamma_k = o(1)$, the difference between G and Bd must be spread across $\Theta(r)$ types, each perturbed below the per-type sampling resolution of a length-k prefix — which is precisely the structure of the *tolerant-testing* hard instances invoked in §B.5. This is also why a low-dimensional construction fails (§B.7).

B.3 The rare-resource cell

The construction is assembled from independent *cells*, each computable in closed form.

The cell. Two offline resources $\{a, b\}$. Arrivals, in order: a *flexible* request F with neighborhood $\{a, b\}$, which always arrives and must be routed *before* the specialist is seen; then at most one *specialist* — type α (neighborhood $\{a\}$) with probability θs, type β (neighborhood $\{b\}$) with probability $\theta(1-s)$, or no specialist with probability $1-\theta$. Here $\theta \in (0, 1]$ is the **contention rate** and $s \in [0, 1]$ the **bias**. The advice specifies a predicted bias $\hat{s}$; following it routes F to protect the predicted-majority specialist ($\hat{s} > \frac{1}{2}$: route $F \to b$, saving a for the likelier α; symmetrically otherwise).

Lemma B.2 (single-cell constants). Per cell, in expectation:

quantity	value
OPT	$1+\theta$
Baseline (route F uniformly)	$1+\theta/2$
Mimic, advice direction *right* ($\text{sign}(\hat{s}-\frac{1}{2})=\text{sign}(s-\frac{1}{2})$)	$1+\theta\max(s,1-s)$
Mimic, advice direction *wrong*	$1+\theta\min(s,1-s)$

Hence the per-cell advantage of following over the baseline is $\pm\,\theta\,|s-\frac{1}{2}|$ (sign $=$ is the advice direction correct), and the ℓ_1 distance between the cell's specialist distribution $p=(\theta s,\ \theta(1-s),\ 1-\theta)$ and the advice $q=(\theta\hat{s},\ \theta(1-\hat{s}),\ 1-\theta)$ is $\ell_1(p,q)=2\theta\,|s-\hat{s}|$.

Proof. Routing $F\to a$ matches F always and the specialist unless it is α (whose only neighbor a is taken): $\theta s\cdot 1+\theta(1-s)\cdot 2+(1-\theta)\cdot 1=1+\theta(1-s)$. Symmetrically $F\to b$ yields $1+\theta s$. The baseline averages the two, $1+\theta/2$. The better route is $F\to b$ iff $s>\frac{1}{2}$, giving $1+\theta\max(s,1-s)$ for the right direction and $1+\theta\min(s,1-s)$ for the wrong one. OPT $=2$ when a specialist arrives (probability θ), else 1, totalling $1+\theta$. The ℓ_1 computation is a three-coordinate sum whose $(1-\theta)$ coordinate cancels. ■

Two quantities separate cleanly, which is the point of the design: the *magnitude* of the advantage, $\theta|s-\frac{1}{2}|$, depends only on the truth's signal (how contested the cell is), while its *sign* depends only on whether the advice's direction is right. Deciding whether the advice is net-helpful is therefore exactly the problem of recovering per-cell *directions* — a Bernoulli discrimination needing $\approx 1/|s-\frac{1}{2}|^2$ samples *of that cell*. (Numerical check, 4×10^5 trials per cell: at $\theta=0.6$, $s=0.7$ the simulated advantage is ± 0.119 against the formula's ± 0.12; all five identities of Lemma B.2 match to three decimals — Appendix A.6.)

B.4 The aggregate construction and the exact affine law

The family. $m=\Theta(n)$ independent cells, each with a *distinct* specialist type-pair (α_i,β_i), so the type support is $r=\Theta(n)$ and each specialist type is seen $O(1)$ times among n arrivals. Cell i has contention θ_i and truth bias s_i with signal $\varepsilon_i=|s_i-\frac{1}{2}|$; the advice predicts, per cell, a direction with the same magnitude ($\hat{s}_i=\frac{1}{2}\pm\varepsilon_i$, correct or wrong side). Because cells are disjoint, per-cell quantities aggregate additively.

Lemma B.3 (exact affine conversion law). Let $\rho_{\text{perfect}}=(\sum_i(1+\theta_i\max(s_i,1-s_i)))/\text{OPT}$ be the all-directions-correct (consistency ceiling) ratio, with $\text{OPT}=\sum_i(1+\theta_i)$. Then, for any advice with per-cell magnitudes matching the truth,

$$\mathbb{E}[\text{follow-ratio}]\ =\ \rho_{\text{perfect}}-\tfrac{1}{2}\,\ell_1(p,q),$$

and hence the break-even advice error is $\ell_1^*=2(\rho_{\text{perfect}}-\rho_{\text{base}})$, where $\rho_{\text{base}}=(\sum_i(1+\theta_i/2))/\text{OPT}$.

Proof. By Lemma B.2, cell i contributes $1+\theta_i\max(s_i,1-s_i)$ to the followed matching if its advice direction is correct and $1+\theta_i\min(s_i,1-s_i)$ if wrong; the difference is $2\theta_i\varepsilon_i$. Letting W be the set of wrong-direction cells,

$$\text{followed matching}=\sum_i(1+\theta_i\max(s_i,1-s_i))-\sum_{i\in W}2\theta_i\varepsilon_i.$$

For the ℓ_1: the flexible and no-specialist coordinates cancel, and cell i contributes $2\theta_i|s_i-\hat{s}_i|$, which is 0 if correct and (opposite side, same magnitude, so $|s_i-\hat{s}_i| = 2\varepsilon_i$) $4\theta_i\varepsilon_i$ if wrong. The type normalizer is $N = \sum_i(1+\theta_i) = \mathrm{OPT}$, so $\ell_1(p,q) = (\sum_{i\in W} 4\theta_i\varepsilon_i)/\mathrm{OPT} = 2(\sum_{i\in W} 2\theta_i\varepsilon_i)/\mathrm{OPT}$. Substituting,

$$\mathbb{E}[\text{follow-ratio}] = \frac{\text{followed matching}}{\mathrm{OPT}} = \rho_{\text{perfect}} - \tfrac{1}{2}\,\ell_1(p,q). \qquad \blacksquare$$

The followed matching is a sum of m independent bounded cell contributions, so it concentrates around its mean at rate $O(1/\sqrt{m})$; the affine law holds with high probability, not only in expectation. (Numerical verification at $\theta = 0.6$, $\varepsilon = 0.3$, $m = 4000$ — $\rho_{\text{perfect}} = 0.924$, $\rho_{\text{base}} = 0.812$ — matches the law to three decimals at every wrong-fraction $\varphi \in \{0, 0.2, 0.5, 0.8, 1\}$, with the simulated break-even 0.223 against the predicted $\ell_1^* = 2(0.924 - 0.812) = 0.224$; Appendix A.6.)

Consequence: the stakes and the gap are clean, and the good side is a ball. Define "following gains $\geq \delta$" $\iff$ $\ell_1 \leq \ell_1^* - 2\delta =: a$ and "following loses $\geq \Delta$" $\iff$ $\ell_1 \geq \ell_1^* + 2\Delta =: b$. Choosing, e.g., $\delta = \Delta = (\rho_{\text{perfect}} - \rho_{\text{base}})/4$ gives $a = \ell_1^*/2 > 0$, $b = 3\ell_1^*/2$, and gap $b - a = \ell_1^*$ — all $\Theta(1)$ constants independent of n (set by the fixed per-cell θ, ε; only the number of cells grows). Crucially $a > 0$: good advice is a $\Theta(1)$-radius *ball* around the truth, not the single point $p = q$. This is what places the decision problem in the *tolerant* testing regime (§B.5).

B.5 Reduction to tolerant identity testing

Lemma B.4 (algorithm ⇒ tester). On the family of §B.4, fix the advice q. Suppose $A \in \mathcal{A}_k$ is $(1-\eta_c)$-consistent whenever $\ell_1(p,q) \leq a$ and loses at most $\eta_r\Delta$ whenever $\ell_1(p,q) \geq b$. Then the decision D of A, viewed as a function of the k i.i.d. samples $X_{1:k} \sim p$, distinguishes $\{\ell_1(p,q) \leq a\}$ from $\{\ell_1(p,q) \geq b\}$ with error at most $\eta_c + \eta_r + o(1)$.

Proof sketch. Consistency forces $D = \mathrm{F}$ with high probability when $\ell_1 \leq a$ (otherwise the algorithm forgoes the δ gain, contradicting $(1-\eta_c)$-consistency); robustness forces $D = \mathrm{R}$ with high probability when $\ell_1 \geq b$ (otherwise it eats the Δ loss). So D itself is the tester, and its error probabilities are bounded by the consistency and robustness slacks via the same conditioning as in Lemma B.1. $\blacksquare$

The key point is that the prefix $X_{1:k}$ is *literally* k i.i.d. samples from the type distribution p (the known-i.i.d. model), and D is an arbitrary function of them — so a sample-complexity lower bound for the testing problem rules out *every* rule in $\mathcal{A}_k$, not only the empirical-ℓ_1 threshold. This is the step that makes the result algorithm-independent and distinguishes it from the constructive baseline-coupling inside Choo et al.'s algorithm (Chapter 9).

Theorem B.5 (tolerant identity testing; cited). Distinguishing $\ell_1(p,q) \leq \varepsilon_1$ from $\ell_1(p,q) \geq \varepsilon_2$ for a known q over support size r, with error $\leq 1/3$, requires

$$k \;=\; \tilde{\Theta}\Big(\frac{\sqrt{r}}{\varepsilon_2^2} \;+\; \frac{r}{\log r}\cdot\max\Big\{\frac{\varepsilon_1}{\varepsilon_2^2}, \Big(\frac{\varepsilon_1}{\varepsilon_2^2}\Big)^2\Big\}\Big)$$

samples [18]. In particular, for constant tolerances $0 < \varepsilon_1 < \varepsilon_2$ the complexity is $\tilde{\Theta}(r/\log r)$ — near-linear — whereas the non-tolerant case $\varepsilon_1 = 0$ needs only $\Theta(\sqrt{r}/\varepsilon_2^2)$ [15, 16]. See also [13, 17] for the $\Omega(r/\log r)$ lower bound at constant tolerance and the ℓ_1-estimation constants.

Because §B.4 gives $a > 0$ (a $\Theta(1)$ ball, not a point), our testing problem has constant tolerances $\varepsilon_1 = a$, $\varepsilon_2 = b$ and falls in the *tolerant* regime: with $r = \Theta(n)$ types, any tester needs $\tilde{\Theta}(n/\log n)$ samples. The $\Theta(\sqrt{r})$ non-tolerant escape does not apply — and this is not an artifact but a design necessity, since demanding consistency only at exactly-correct advice ($a = 0$) would make the consistency guarantee vacuous.

B.6 Assembling the theorem

Chaining §B.2–§B.5 on the family of §B.4 with constant θ, ε (hence $\rho_{\text{base}} = 1 - \Theta(1)$, stakes $\delta, \Delta = \Theta(1)$, tolerances $a, b = \Theta(1)$, support $r = \Theta(n)$):

1. Suppose $A \in \mathcal{A}_k$ with $k = o(n/\log n)$ is $(1 - o(1))$-consistent and $(\rho_{\text{base}} - o(1))$-robust on the family.
2. By Lemma B.4, its decision solves the (a, b)-tolerant identity-testing problem over support $\Theta(n)$ with error $o(1)$ from k samples.
3. By Theorem B.5, any such tester needs $\tilde{\Theta}(n/\log n)$ samples — more than k provides. The witness pair guaranteed by the lower bound consists of two type distributions p_G (at $\ell_1(p_G, q) \le a$) and p_{Bd} (at $\ell_1(p_{\text{Bd}}, q) \ge b$) whose length-$k$ prefix laws satisfy $\gamma_k = o(1)$; by the affine law (Lemma B.3), *any* pair at those distances realizes the gain δ and loss Δ respectively.
4. Lemma B.1 applied to (p_G, p_{Bd}) with $\gamma_k = o(1)$ forces $\eta_c + \eta_r \ge 1 - o(1)$ — contradicting step 1.

Theorem B.6 (impossibility; the theorem of Chapter 9). For every test budget $k = o(n/\log n)$ there is a known-i.i.d. matching family with $r = \Theta(n)$ types and baseline strength $\rho_{\text{base}} = 1 - \Theta(1)$ on which no test-and-fallback algorithm $A \in \mathcal{A}_k$ — deciding by any rule on its prefix — is simultaneously $(1 - o(1))$-consistent and $(\rho_{\text{base}} - o(1))$-robust. The achievable (consistency, robustness) region collapses to the baseline point $(\rho_{\text{base}}, \rho_{\text{base}})$.

Combined with the easy strong-baseline side — few types ⇒ near-perfect matching ⇒ upside $\delta \to 0$, nothing to capture — this is the scissors of Figure 9.1: the upside exists only where the tolerant test is infeasible.

B.7 Status, the remaining step, and a negative finding

Proved here. Lemma B.1 (complete proof, §B.2); Lemma B.2 (complete proof plus three-decimal numerical verification, §B.3); Lemma B.3 (complete proof plus verification, §B.4); Lemma B.4 (proof sketch whose completion is bookkeeping on the two conditioning displays of Lemma B.1). Theorem B.5 is a cited, exact, modern result — the load-bearing external ingredient — and §B.4 establishes that our problem sits on its tolerant (theorem-preserving) side.

The remaining step (stated plainly). Step 3 of §B.6 requires *exhibiting* the witness pair (p_G, p_{Bd}) from the tolerant-testing lower-bound construction and checking it lands at $\ell_1 \le a$ and $\ell_1 \ge b$ within the cell parameterization of §B.4. Because the affine law makes any pair at those distances realize the stakes, this is a routine verification rather than new mathematics; it, plus a final end-to-end review, remains open at the time of writing (see §9.5). Until it is closed, Theorem B.6 should be read with the status stated in Chapter 9.

A negative finding worth recording. A *low-dimensional* version of the construction — all cells

sharing one global bias parameter, with the good/bad scenarios differing by a single global shift — provably fails: a length-k prefix estimates one shared parameter to $\pm 1/\sqrt{\theta k}$, so the scenarios become distinguishable at any $\delta = \omega(\sqrt{\theta/k})$ and only a vacuous impossibility survives. The strength of the theorem genuinely requires spreading the advice error across $\Theta(n)$ independent cells, each perturbed below the per-type sampling resolution — the dimension is essential, which is also why the theorem's strong form is stated for the $r = \Theta(n)$ regime and a constant-r version is left open (§9.5).